```
console.log('灵宠消消塔开始初始化...')
// 平台检测（game.js 在 require 之前执行，需内联检测）
const P = (typeof tt !== 'undefined') ? tt : wx
GameGlobal.__platform = P
// 侧边栏复访（抖音必接）：必须在 game.js 运行时机同步监听 onShow
GameGlobal.__launchInfo = {}
GameGlobal.__sidebarSupported = false
if (typeof P.onShow === 'function') {
  P.onShow(function (res) {
    console.log('[Sidebar] onShow:', JSON.stringify(res))
    GameGlobal.__launchInfo = res || {}
  })
}
if (typeof P.checkScene === 'function') {
  P.checkScene({
    scene: 'sidebar',
    success: function (res) {
      GameGlobal.__sidebarSupported = !!(res && res.isExist)
      console.log('[Sidebar] checkScene supported:', GameGlobal.__sidebarSupported)
    },
    fail: function () { GameGlobal.__sidebarSupported = false }
  })
}
// 兼容：抖音无 getWindowInfo，用 getSystemInfoSync 代替
function _getWinInfo() {
  if (typeof P.getWindowInfo === 'function') return P.getWindowInfo()
  const s = P.getSystemInfoSync()
  return { windowWidth: s.windowWidth || s.screenWidth, windowHeight: s.windowHeight || s.screenH...
}
// 创建主Canvas并挂到全局，供main.js复用
const _canvas = P.createCanvas()
GameGlobal.__mainCanvas = _canvas
// 立即填充背景色，然后加载主包内的 loading 背景图覆盖上去
const _ctx = _canvas.getContext('2d')
const _winInfo = _getWinInfo()
const _dpr = _winInfo.pixelRatio || 2
_canvas.width = _winInfo.windowWidth * _dpr
_canvas.height = _winInfo.windowHeight * _dpr
const _W = _canvas.width, _H = _canvas.height
// 先画纯色兜底
const _bg = _ctx.createLinearGradient(0, 0, 0, _H)
_bg.addColorStop(0, '#0b0b15')
_bg.addColorStop(1, '#1a1035')
_ctx.fillStyle = _bg
_ctx.fillRect(0, 0, _W, _H)
// 加载主包内的 loading 背景图（不依赖分包）
const _splashImg = P.createImage()
_splashImg.src = 'loading_bg.jpg'
_splashImg.onload = () => {
  // 铺满整个 canvas（cover 模式）
```

```
51  const iw = _splashImg.width, ih = _splashImg.height
52  const scale = Math.max(_W / iw, _H / ih)
53  const dw = iw * scale, dh = ih * scale
54  _ctx.drawImage(_splashImg, (_W - dw) / 2, (_H - dh) / 2, dw, dh)
55 }
56 // 分包加载
57 const _subPkgNames = [
58  'pets', 'enemies', 'backgrounds', 'ui', 'hero',
59  'share', 'equipment', 'battle', 'orbs', 'intro',
60  'audio', 'audio_bgm'
61 ]
62 let _loadedCount = 0
63 const _totalPkgs = _subPkgNames.length
64 function _onPkgDone(name, ok) {
65  console.log('[SubPkg] ' + name + (ok ? ' 加载成功' : ' 加载失败'))
66  _loadedCount++
67  if (_loadedCount >= _totalPkgs) {
68   console.log('[SubPkg] 全部 ' + _totalPkgs + ' 个分包加载完成，启动游戏')
69   try {
70    require('./js/main')
71    console.log('游戏初始化成功')
72   } catch (e) {
73    console.error('游戏初始化失败:', e)
74    const ctx = _canvas.getContext('2d')
75    ctx.fillStyle = '#000'; ctx.fillRect(0, 0, _canvas.width, _canvas.height)
76    ctx.fillStyle = '#f00'; ctx.font = '16px sans-serif'
77    ctx.textAlign = 'left'; ctx.textBaseline = 'top'
78    ctx.fillText('游戏初始化失败', 20, 20)
79    ctx.fillText(e.message, 20, 50)
80    ctx.fillText(e.stack ? e.stack.substring(0, 200) : '', 20, 80)
81    P.showModal({ title: '初始化失败', content: e.message, showCancel: false })
82   }
83  }
84 }
85 _subPkgNames.forEach(function (name) {
86  P.loadSubpackage({
87   name: name,
88   success: function () { _onPkgDone(name, true) },
89   fail: function () { _onPkgDone(name, false) }
90  })
91 })
92 /**
93  * 统一后端 API 客户端
94  * 小游戏端通过 P.request (wx.request / tt.request) 与后端通信
95  */
96 const P = require('./platform')
97 // 后端地址：上线后替换为正式域名
98 const BASE_URL = P.isDouyin
99  ? 'https://1lujade2yoizo-env-eredrLayEN.service.douyincloud.run'
100  : 'https://your-wechat-backend.com'
```

```
let _token = ''
function _request(method, path, data) {
  return new Promise((resolve, reject) => {
    P.request({
      url: BASE_URL + path,
      method,
      header: {
        'Content-Type': 'application/json',
        ...._token ? { Authorization: 'Bearer ' + _token } : {},
      },
      data,
      success: (res) => {
        if (res.statusCode === 200 && res.data && res.data.code === 0) {
          resolve(res.data)
        } else {
          reject(new Error((res.data && res.data.msg) || 'request failed'))
        }
      },
      fail: (err) => reject(new Error(err.errMsg || 'network error')),
    })
  })
}
const api = {
  /**
   * 登录 — 获取 token 供后续请求使用
   * 抖音云部署时请求头自动带 x-tt-openid，可跳过此步
   */
  login() {
    return new Promise((resolve, reject) => {
      P.login({
        success: async (loginRes) => {
          try {
            const result = await _request('POST', '/api/login', {
              platform: P.name,
              code: loginRes.code,
            })
            _token = result.token
            console.log('[API] 登录成功, platform=', P.name)
            resolve(result)
          } catch (e) {
            console.warn('[API] 登录换 token 失败，尝试无 token 模式:', e.message)
            _token = ''
            resolve({ code: 0, msg: 'fallback' })
          }
        },
        fail: (err) => {
          console.warn('[API] P.login 失败:', err.errMsg || err)
          _token = ''
          resolve({ code: 0, msg: 'login skipped' })
        },
```

```
      })
    })
  },
  getPlayerData() {
    return _request('GET', '/api/player/data')
  },
  syncPlayerData(data) {
    return _request('POST', '/api/player/sync', { data })
  },
  submitRanking(data) {
    return _request('POST', '/api/ranking/submit', data)
  },
  getRankingList(tab, limit) {
    tab = tab || 'all'
    limit = limit || 50
    return _request('GET', `/api/ranking/list?tab=${tab}&limit=${limit}`)
  },
  get hasToken() { return !!_token },
}
module.exports = api
/**
 * 战斗辅助函数 — 从 Main 中提取的布局计算、动画触发、伤害处理
 * 通过 ViewEnv 获取屏幕常量，避免依赖 Main 的闭包变量
 */
const { ATTR_COLOR } = require('./data/tower')
const { TH } = require('./render')
const MusicMgr = require('./runtime/music')
const V = require('./views/env')
function getBattleLayout() {
  const { W, H, S, safeTop, COLS, ROWS } = V
  const boardPad = 6*S, cellSize = (W-boardPad*2)/COLS, boardH = ROWS*cellSize
  const boardTop = H-8*S-boardH
  const sidePad = 8*S, petGap = 8*S, wpnGap = 12*S
  const totalGapW = wpnGap + petGap * 4 + sidePad * 2
  const iconSize = (W - totalGapW) / 6
  const teamBarH = iconSize + 6*S
  const hpBarH = 18*S
  const hpBarY = boardTop - hpBarH - 4*S
  const teamBarY = hpBarY - teamBarH - 2*S
  const eAreaTop = safeTop + 4*S
  return { boardPad, cellSize, boardH, boardTop, iconSize, teamBarH, teamBarY, hpBarY, eAreaTop }
}
function getEnemyCenterY() {
  const L = getBattleLayout()
  const eAreaBottom = L.teamBarY - 4*V.S
  const eAreaH = eAreaBottom - L.eAreaTop
  return L.eAreaTop + eAreaH * 0.42
}
function playHeroAttack(g, skillName, attr, type) {
  g.heroAttackAnim = { active:true, progress:0, duration:24 }
```

```
  g.enemyHurtAnim  = { active:true, progress:0, duration:18 }
  g._enemyHitFlash = 12
  g._enemyTintFlash = 8
  // 顿帧：命中时短暂冻结增强打击感
  g._hitStopFrames = 3
  // 屏幕闪白
  g._screenFlash = 4; g._screenFlashMax = 4; g._screenFlashColor = '#fff'
  const color = (ATTR_COLOR[attr] && ATTR_COLOR[attr].main) || TH.accent
  const eCenterY = getEnemyCenterY()
  g.skillCastAnim = { active:true, progress:0, duration:30, type:type||'slash', color, skillName:...
}
function playEnemyAttack(g) {
  g.enemyAttackAnim = { active:true, progress:0, duration:20 }
  g.heroHurtAnim    = { active:true, progress:0, duration:18 }
  g._hitStopFrames = 2
  // 受击闪红
  g._screenFlash = 3; g._screenFlashMax = 3; g._screenFlashColor = '#ff2244'
  g._heroHurtFlash = 10
  const L = getBattleLayout()
  g.skillCastAnim = { active:true, progress:0, duration:30, type:'enemyAtk', color:TH.danger, ski...
}
function playHealEffect(g) {
  const L = getBattleLayout()
  g.skillCastAnim = { active:true, progress:0, duration:25, type:'heal', color:'#4dcc4d', skillNa...
  MusicMgr.playHeal()
}
/** 统一添加护盾（自动应用法宝 shieldBoost 加成） */
function addShield(g, val) {
  if (g.weapon && g.weapon.type === 'shieldBoost') {
    val = Math.round(val * (1 + (g.weapon.pct || 50) / 100))
  }
  g.heroShield += val
  MusicMgr.playShieldGain()
  g.dmgFloats.push({ x:V.W*0.5, y:V.H*0.65, text:`+${val}盾`, color:'#7ddfff', t:0, alpha:1 })
}
/** 对英雄造成伤害（含护盾、绝对防御、飘字） */
function dealDmgToHero(g, dmg) {
  const immune = g.heroBuffs && g.heroBuffs.find(b => b.type === 'dmgImmune')
  if (immune && dmg > 1) dmg = 1
  const { W, H } = V
  if (g.heroShield > 0) {
    if (dmg <= g.heroShield) {
      g.heroShield -= dmg
      g.skillEffects.push({ x:W*0.5, y:H*0.52, text:'完美抵挡！', color:'#40e8ff', t:0, alpha:1, scale...
      g.dmgFloats.push({ x:W*0.5, y:H*0.6, text:`护盾吸收 ${dmg}`, color:'#7ddfff', t:0, alpha:1, sca...
      g.shakeT = 4; g.shakeI = 2
      g._blockFlash = 8
      MusicMgr.playBlock()
      return
    }
```

```
    const shieldAbs = g.heroShield
    dmg -= g.heroShield; g.heroShield = 0
    g.skillEffects.push({ x:W*0.5, y:H*0.52, text:'护盾击碎！', color:'#ff9040', t:0, alpha:1, scale:2...
    g.dmgFloats.push({ x:W*0.45, y:H*0.6, text:`盾挡 ${shieldAbs}`, color:'#40b8e0', t:0, alpha:1, ...
  }
  const oldPct = g.heroHp / g.heroMaxHp
  g.heroHp = Math.max(0, g.heroHp - dmg)
  g._heroHpLoss = { fromPct: oldPct, timer: 0 }
  g.dmgFloats.push({ x:W*0.5, y:H*0.7, text:`-${dmg}`, color:TH.danger, t:0, alpha:1 })
}
module.exports = { getBattleLayout, getEnemyCenterY, playHeroAttack, playEnemyAttack, playHealEff...
/**
 * 宝箱里程碑奖励配置
 *
 * 每条里程碑包含:
 *  - id: 唯一标识
 *  - type: 条件类型 (level / bestFloor / totalRuns / petPoolCount)
 *  - threshold: 达成阈值
 *  - name: 显示名称
 *  - rewards[]: 可扩展奖励数组
 *
 * 奖励 type:
 *  - fragment: 碎片（按 tierWeights 随机分配给某只宠物）
 *  - pet:     直送宠物入池（从指定档位未拥有宠物中随机选）
 *  - exp:     修炼经验
 *  - petExp:  宠物经验（共享池）
 *  - stamina: 体力
 *  - item:    道具（预留）
 */
const { PET_TIER, getPetTier } = require('./pets')
// 碎片召唤费用（按档位）
const SUMMON_FRAG_COST = { T3: 10, T2: 15, T1: 25 }
// ===== 里程碑定义 =====
const CHEST_MILESTONES = [
  // —— 修炼等级（Lv2起，之后每级，之后每5级）——
  { id: 'lv_2',  type: 'level', threshold: 2,  name: '修炼 Lv.2',
    desc: '漫漫修仙路，从升第一级开始，灵宠们为你欢呼！',
    rewards: [
      { type: 'fragment', count: 6,  tierWeights: { T3: 85, T2: 15, T1: 0 } },
      { type: 'petExp', amount: 60 },
    ]},
  { id: 'lv_3',  type: 'level', threshold: 3,  name: '修炼 Lv.3',
    desc: '感气三重，灵力渐聚，天地灵气开始向你汇聚。',
    rewards: [
      { type: 'fragment', count: 8,  tierWeights: { T3: 85, T2: 15, T1: 0 } },
      { type: 'petExp', amount: 100 },
    ]},
  { id: 'lv_4',  type: 'level', threshold: 4,  name: '修炼 Lv.4',
    desc: '感气四重，功法愈发纯熟，灵宠也感受到你的进步。',
    rewards: [
```

```
    { type: 'fragment', count: 8,  tierWeights: { T3: 80, T2: 20, T1: 0 } },
    { type: 'petExp', amount: 80 },
  ]},
{ id: 'lv_5',  type: 'level', threshold: 5,  name: '练气期突破',
  desc: '第一道天堑已破！境界提升，新修炼形象「剑灵少侠」已解锁！',
  rewards: [
    { type: 'avatar', avatarId: 'boy2' },
    { type: 'fragment', count: 10, tierWeights: { T3: 75, T2: 25, T1: 0 } },
    { type: 'petExp', amount: 150 },
    { type: 'stamina', amount: 30 },
  ]},
{ id: 'lv_6',  type: 'level', threshold: 6,  name: '修炼 Lv.6',
  desc: '修为日进，灵宠们纷纷仰望，你散发出不一样的气息。',
  rewards: [
    { type: 'fragment', count: 8,  tierWeights: { T3: 70, T2: 30, T1: 0 } },
    { type: 'petExp', amount: 100 },
  ]},
{ id: 'lv_7',  type: 'level', threshold: 7,  name: '修炼 Lv.7',
  desc: '越来越熟练了，同辈修士们开始侧目，你走在他们前面。',
  rewards: [
    { type: 'fragment', count: 8,  tierWeights: { T3: 65, T2: 30, T1: 5 } },
    { type: 'petExp', amount: 120 },
  ]},
{ id: 'lv_8',  type: 'level', threshold: 8,  name: '修炼 Lv.8',
  desc: '感气八重，离筑基只差一步，再努力一把！',
  rewards: [
    { type: 'fragment', count: 10, tierWeights: { T3: 60, T2: 35, T1: 5 } },
    { type: 'petExp', amount: 150 },
  ]},
{ id: 'lv_9',  type: 'level', threshold: 9,  name: '修炼 Lv.9',
  desc: '感气大成！境界已触碰极限，下一步将是质的飞跃。',
  rewards: [
    { type: 'fragment', count: 8,  tierWeights: { T3: 55, T2: 35, T1: 10 } },
    { type: 'petExp', amount: 120 },
  ]},
{ id: 'lv_10', type: 'level', threshold: 10, name: '修炼 Lv.10',
  desc: '感气圆满！宗门嘉奖，新修炼形象「星月仙子」已解锁！',
  rewards: [
    { type: 'avatar', avatarId: 'girl2' },
    { type: 'fragment', count: 12, tierWeights: { T3: 50, T2: 40, T1: 10 } },
    { type: 'petExp', amount: 200 },
    { type: 'pet', tier: 'T3' },
  ]},
{ id: 'lv_15', type: 'level', threshold: 15, name: '筑基期突破',
  desc: '凝气成丹，根基铸就，蜕变已然开始，更广阔的天地在等你。',
  rewards: [
    { type: 'fragment', count: 15, tierWeights: { T3: 40, T2: 45, T1: 15 } },
    { type: 'petExp', amount: 300 },
    { type: 'stamina', amount: 50 },
  ]},
```

```
{ id: 'lv_20', type: 'level', threshold: 20, name: '修炼 Lv.20',
 desc: 'Lv.20，你的名字已在修仙界小有传闻，颇有几分传奇色彩。',
 rewards: [
   { type: 'fragment', count: 18, tierWeights: { T3: 30, T2: 50, T1: 20 } },
   { type: 'petExp', amount: 400 },
 ]},
{ id: 'lv_25', type: 'level', threshold: 25, name: '修炼 Lv.25',
 desc: '历经风雨，依然坚持，Lv.25是你不曾放弃的最好见证。',
 rewards: [
   { type: 'fragment', count: 20, tierWeights: { T3: 20, T2: 50, T1: 30 } },
   { type: 'petExp', amount: 600 },
 ]},
{ id: 'lv_30', type: 'level', threshold: 30, name: '金丹期突破',
 desc: '炼就金丹！这一刻，你真正踏入了修仙之路的核心，传说从此开始。',
 rewards: [
   { type: 'fragment', count: 25, tierWeights: { T3: 10, T2: 50, T1: 40 } },
   { type: 'petExp', amount: 800 },
   { type: 'stamina', amount: 50 },
   { type: 'pet', tier: 'T2' },
 ]},
{ id: 'lv_35', type: 'level', threshold: 35, name: '修炼 Lv.35',
 desc: '登高方知山高路远，Lv.35的你已是同辈中的佼佼者。',
 rewards: [
   { type: 'fragment', count: 22, tierWeights: { T3: 10, T2: 45, T1: 45 } },
   { type: 'petExp', amount: 800 },
 ]},
{ id: 'lv_40', type: 'level', threshold: 40, name: '修炼 Lv.40',
 desc: 'Lv.40，你已超越了大多数修士，灵宠们以跟随你为荣。',
 rewards: [
   { type: 'fragment', count: 25, tierWeights: { T3: 5, T2: 40, T1: 55 } },
   { type: 'petExp', amount: 1000 },
 ]},
{ id: 'lv_45', type: 'level', threshold: 45, name: '元婴期突破',
 desc: '神识独立，元婴出窍，超脱肉身束缚，你已触摸到永生的边缘！',
 rewards: [
   { type: 'fragment', count: 30, tierWeights: { T3: 0, T2: 35, T1: 65 } },
   { type: 'petExp', amount: 1500 },
   { type: 'stamina', amount: 50 },
 ]},
{ id: 'lv_50', type: 'level', threshold: 50, name: '修炼 Lv.50',
 desc: 'Lv.50，这方天地的强者之列已有你的一席之地，传说不虚。',
 rewards: [
   { type: 'fragment', count: 30, tierWeights: { T3: 0, T2: 30, T1: 70 } },
   { type: 'petExp', amount: 1500 },
 ]},
{ id: 'lv_55', type: 'level', threshold: 55, name: '修炼 Lv.55',
 desc: '化神期已近在眼前，Lv.55的你令众仙惊叹，终点即将到来。',
 rewards: [
   { type: 'fragment', count: 35, tierWeights: { T3: 0, T2: 25, T1: 75 } },
   { type: 'petExp', amount: 2000 },
```

```
  ]},
 { id: 'lv_58', type: 'level', threshold: 58, name: '化神期突破',
  desc: '化形为神，名列仙籍！你的名字将永远刻在修仙界的历史上。',
  rewards: [
    { type: 'fragment', count: 40, tierWeights: { T3: 0, T2: 20, T1: 80 } },
    { type: 'petExp', amount: 2500 },
    { type: 'pet', tier: 'T1' },
  ]},
 { id: 'lv_60', type: 'level', threshold: 60, name: '修炼圆满',
  desc: '大道已成，修炼圆满！天地为证，所有灵宠与你共享这份荣耀！',
  rewards: [
    { type: 'fragment', count: 50, tierWeights: { T3: 0, T2: 15, T1: 85 } },
    { type: 'petExp', amount: 3000 },
    { type: 'stamina', amount: 100 },
  ]},
 // —— 肉鸽塔层数 ——
 { id: 'tower_5',  type: 'bestFloor', threshold: 5,  name: '通天塔 5 层',
  desc: '打到第5层了！灵宠们兴奋地围着你蹦跳，不错的开始！',
  rewards: [
    { type: 'fragment', count: 6,  tierWeights: { T3: 90, T2: 10, T1: 0 } },
    { type: 'petExp', amount: 60 },
  ]},
 { id: 'tower_10', type: 'bestFloor', threshold: 10, name: '通天塔 10 层',
  desc: '第10层！越打越猛，你和灵宠之间的默契越来越深。',
  rewards: [
    { type: 'fragment', count: 10, tierWeights: { T3: 70, T2: 25, T1: 5 } },
    { type: 'petExp', amount: 150 },
  ]},
 { id: 'tower_15', type: 'bestFloor', threshold: 15, name: '通天塔 15 层',
  desc: '过了半塔！前方未知，但你已证明自己远超常人。',
  rewards: [
    { type: 'fragment', count: 12, tierWeights: { T3: 50, T2: 35, T1: 15 } },
    { type: 'petExp', amount: 200 },
  ]},
 { id: 'tower_20', type: 'bestFloor', threshold: 20, name: '通天塔 20 层',
  desc: '第20层！你的名字已被刻在塔壁上，后来者仰望的传说。',
  rewards: [
    { type: 'fragment', count: 18, tierWeights: { T3: 30, T2: 45, T1: 25 } },
    { type: 'petExp', amount: 400 },
  ]},
 { id: 'tower_25', type: 'bestFloor', threshold: 25, name: '通天塔 25 层',
  desc: '接近塔顶！气势如虹，连塔中的远古妖兽都开始颤抖。',
  rewards: [
    { type: 'fragment', count: 20, tierWeights: { T3: 15, T2: 45, T1: 40 } },
    { type: 'petExp', amount: 500 },
  ]},
 { id: 'tower_30', type: 'bestFloor', threshold: 30, name: '通天塔登顶',
  desc: '绝世强者！登顶通天塔，所有灵宠在你面前俯首称臣！',
  rewards: [
    { type: 'fragment', count: 30, tierWeights: { T3: 0, T2: 40, T1: 60 } },
```

```
    { type: 'petExp', amount: 800 },
    { type: 'pet', tier: 'T2' },
  ]},
// —— 挑战次数 ——
{ id: 'runs_1',  type: 'totalRuns', threshold: 1,  name: '首次冒险',
  desc: '第一次完成挑战！ 灵宠们欢呼雀跃，修仙冒险路正式开启！ ',
  rewards: [
    { type: 'fragment', count: 5,  tierWeights: { T3: 100, T2: 0, T1: 0 } },
    { type: 'pet', tier: 'T3' },
  ]},
{ id: 'runs_3',  type: 'totalRuns', threshold: 3,  name: '初露锋芒',
  desc: '完成了3次挑战，棒棒的！ 渐入佳境，感觉越来越顺手了。',
  rewards: [
    { type: 'fragment', count: 6,  tierWeights: { T3: 90, T2: 10, T1: 0 } },
    { type: 'petExp', amount: 80 },
  ]},
{ id: 'runs_5',  type: 'totalRuns', threshold: 5,  name: '身经百战',
  desc: '完成了5次挑战！ 你已有了自己的战斗风格，灵宠们以你为荣。',
  rewards: [
    { type: 'fragment', count: 8,  tierWeights: { T3: 80, T2: 20, T1: 0 } },
    { type: 'petExp', amount: 100 },
  ]},
{ id: 'runs_10', type: 'totalRuns', threshold: 10, name: '老练冒险者',
  desc: '10次挑战，经验满满！ 你已是宗门里公认的老练冒险者了。',
  rewards: [
    { type: 'fragment', count: 12, tierWeights: { T3: 60, T2: 30, T1: 10 } },
    { type: 'petExp', amount: 200 },
  ]},
{ id: 'runs_20', type: 'totalRuns', threshold: 20, name: '百折不挠',
  desc: '完成了20次挑战！ 百折不挠是对你最好的注脚，堪称楷模。',
  rewards: [
    { type: 'fragment', count: 15, tierWeights: { T3: 40, T2: 40, T1: 20 } },
    { type: 'petExp', amount: 300 },
  ]},
// —— 灵宠收集 ——
{ id: 'pool_3',  type: 'petPoolCount', threshold: 3,  name: '初建灵宠池',
  desc: '3只灵宠齐聚！ 你的队伍初具规模，合作的力量已初现端倪。',
  rewards: [
    { type: 'fragment', count: 8,  tierWeights: { T3: 80, T2: 20, T1: 0 } },
    { type: 'petExp', amount: 100 },
  ]},
{ id: 'pool_5',  type: 'petPoolCount', threshold: 5,  name: '灵宠成队',
  desc: '5只灵宠成队！ 五行属性初聚，有了这群伙伴，谁还能挡住你？ ',
  rewards: [
    { type: 'fragment', count: 12, tierWeights: { T3: 60, T2: 30, T1: 10 } },
    { type: 'petExp', amount: 200 },
    { type: 'stamina', amount: 30 },
  ]},
{ id: 'pool_10', type: 'petPoolCount', threshold: 10, name: '灵宠大师',
  desc: '10只灵宠入池！ 这已是一支正规军，同辈修士闻风而逃吧。',
```

```
    rewards: [
      { type: 'fragment', count: 20, tierWeights: { T3: 30, T2: 45, T1: 25 } },
      { type: 'petExp', amount: 400 },
    ]},
  { id: 'pool_15', type: 'petPoolCount', threshold: 15, name: '灵宠收藏家',
    desc: '15只灵宠，你的灵宠池令整个修仙界的同辈都为之羡慕。',
    rewards: [
      { type: 'fragment', count: 25, tierWeights: { T3: 15, T2: 45, T1: 40 } },
      { type: 'petExp', amount: 600 },
    ]},
  { id: 'pool_20', type: 'petPoolCount', threshold: 20, name: '万灵统御',
    desc: '20只灵宠！万灵归心，你已是当之无愧的传说级灵宠大师！',
    rewards: [
      { type: 'fragment', count: 35, tierWeights: { T3: 5, T2: 40, T1: 55 } },
      { type: 'petExp', amount: 800 },
    ]},
]
// ===== 条件检查 =====
function checkCondition(milestone, storage) {
  switch (milestone.type) {
    case 'level':       return (storage.cultivation.level || 0) >= milestone.threshold
    case 'bestFloor':    return (storage.bestFloor || 0) >= milestone.threshold
    case 'totalRuns':    return (storage.totalRuns || 0) >= milestone.threshold
    case 'petPoolCount': return (storage.petPoolCount || 0) >= milestone.threshold
    default: return false
  }
}
/**
 * 返回已达成但未领取的里程碑列表
 */
function getUnclaimedChests(storage) {
  const claimed = (storage._d || storage).chestRewards
    ? ((storage._d || storage).chestRewards.claimed || {})
    : {}
  return CHEST_MILESTONES.filter(m => !claimed[m.id] && checkCondition(m, storage))
}
/**
 * 返回未领取宝箱数量
 */
function getUnclaimedCount(storage) {
  return getUnclaimedChests(storage).length
}
/**
 * 按 tierWeights 随机选一只宠物，集中分配碎片
 * @param {object} tierWeights - { T3: 80, T2: 20, T1: 0 }
 * @returns {string} petId
 */
function rollPetByTier(tierWeights) {
  const totalW = (tierWeights.T3 || 0) + (tierWeights.T2 || 0) + (tierWeights.T1 || 0)
  if (totalW <= 0) return PET_TIER.T3[0]
```

```
  let roll = Math.random() * totalW
  let tier = 'T3'
  if (roll < (tierWeights.T1 || 0)) tier = 'T1'
  else if (roll < (tierWeights.T1 || 0) + (tierWeights.T2 || 0)) tier = 'T2'
  const candidates = PET_TIER[tier]
  if (!candidates || candidates.length === 0) return PET_TIER.T3[0]
  return candidates[Math.floor(Math.random() * candidates.length)]
}
/**
 * 从指定档位中随机选一只未拥有的宠物
 * @returns {string|null} petId or null if all owned
 */
function rollUnownedPet(storage, tier) {
  const pool = (storage._d ? storage._d.petPool : storage.petPool) || []
  const ownedIds = new Set(pool.map(p => p.id))
  const candidates = (PET_TIER[tier] || []).filter(id => !ownedIds.has(id))
  if (candidates.length === 0) return null
  return candidates[Math.floor(Math.random() * candidates.length)]
}
module.exports = {
  SUMMON_FRAG_COST,
  CHEST_MILESTONES,
  checkCondition,
  getUnclaimedChests,
  getUnclaimedCount,
  rollPetByTier,
  rollUnownedPet,
}
/**
 * 云同步模块 — 灵宠消消塔
 * 从 Storage 中拆分出的云端数据同步逻辑
 * 管理：微信云数据库 / 抖音 HTTP API 的双端同步
 */
const P = require('../platform')
const api = require('../api')
const {
  CLOUD_SYNC_BASE_DELAY_MS,
  CLOUD_SYNC_MAX_BACKOFF_MS,
  CLOUD_SYNC_MAX_FAIL,
  CLOUD_SYNC_LOG_THRESHOLD,
  CLOUD_SYNC_RETRY_INTERVAL_MS,
} = require('./constants')
const CLOUD_ENV = 'cloud1-6g8y0x2i39e768eb'
// ===== 内部状态 =====
let _cloudReady = false
let _openid = ''
let _cloudSyncTimer = null
let _cloudInitDone = false
let _pendingSync = false
let _syncFailCount = 0
```

```
601 let _syncDisabled = false
602 let _syncDirty = false
603 let _syncRetryTimer = null
604 let _syncing = false
605 let _syncPending = false
606 // init 时传入的引用和配置
607 let _dataRef = null
608 let _localKey = ''
609 let _currentVersion = 0
610 let _runMigrations = null
611 // ===== 深度合并 =====
612 // cloud 的值覆盖 target，但对嵌套对象递归合并
613 // 保留 target 中有但 cloud 中没有的字段（如后来新增的 bestTotalTurns）
614 function _deepMerge(target, source) {
615   for (const key of Object.keys(source)) {
616     const sv = source[key]
617     const tv = target[key]
618     if (sv && typeof sv === 'object' && !Array.isArray(sv) && tv && typeof tv === 'object' && !Ar...
619       _deepMerge(tv, sv)
620     } else {
621       if (key === 'bestFloor') {
622         target[key] = Math.max(tv || 0, sv || 0)
623       } else if (key === 'bestTotalTurns') {
624         if ((tv || 0) > 0 && (sv || 0) > 0) target[key] = Math.min(tv, sv)
625         else target[key] = (tv || 0) > 0 ? tv : (sv || 0)
626       } else if (key === 'maxCombo') {
627         target[key] = Math.max(tv || 0, sv || 0)
628       } else if (key === 'totalBattles' || key === 'totalCombos' || key === 'totalRuns') {
629         target[key] = Math.max(tv || 0, sv || 0)
630       } else {
631         target[key] = sv
632       }
633     }
634   }
635 }
636 // ===== 从云端拉取并合并 =====
637 async function _syncFromCloud() {
638   if (!_cloudReady) return
639   try {
640     let cloudData = null
641     if (P.isWeChat) {
642       if (!_openid) return
643       const db = P.cloud.database()
644       const res = await db.collection('playerData').where({ _openid: _openid }).get()
645       if (res.data && res.data.length > 0) {
646         cloudData = res.data[0]
647         delete cloudData._id
648         delete cloudData._openid
649       }
650     } else {
```

```
    const res = await api.getPlayerData()
    if (res && res.data) cloudData = res.data
  }
  if (cloudData) {
    const cloudTime = cloudData._updateTime || cloudData.updatedAt || 0
    const localTime = _dataRef._updateTime || 0
    if (cloudTime > localTime) {
      _deepMerge(_dataRef, cloudData)
      if ((_dataRef._version || 0) < _currentVersion) {
        _runMigrations(_dataRef)
      }
      P.setStorageSync(_localKey, JSON.stringify(_dataRef))
      console.log('[Storage] 云端数据已合并到本地')
    }
  }
 } catch(e) { console.warn('Sync from cloud error:', e) }
}
// ===== 推送到云端 =====
async function _syncToCloud() {
 if (!_cloudReady || _syncDisabled) return
 if (_syncing) {
  _syncPending = true
  return
 }
 _syncing = true
 try {
  if (P.isDouyin) {
    await api.syncPlayerData({ ..._dataRef, _updateTime: Date.now() })
  } else {
    if (!_openid) return
    const db = P.cloud.database()
    const col = db.collection('playerData')
    const res = await col.where({ _openid: _openid }).get()
    const saveData = { ..._dataRef, _updateTime: Date.now() }
    delete saveData._id
    delete saveData._openid
    const _ = db.command
    const setData = {}
    for (const k of Object.keys(saveData)) { setData[k] = _.set(saveData[k]) }
    if (res.data && res.data.length > 1) {
      await col.doc(res.data[0]._id).update({ data: setData })
      for (let i = 1; i < res.data.length; i++) {
        try { await col.doc(res.data[i]._id).remove() } catch(e) {}
      }
      console.log('[Storage] 云同步完成，清理了', res.data.length - 1, '条重复记录')
    } else if (res.data && res.data.length === 1) {
      await col.doc(res.data[0]._id).update({ data: setData })
    } else {
      await col.add({ data: saveData })
    }
```

```
    }
    if (_syncFailCount > 0) {
      console.log('[Storage] 云同步恢复成功，已上传最新数据')
    }
    _syncFailCount = 0
    _syncDirty = false
    _syncDisabled = false
    if (_syncRetryTimer) {
      clearInterval(_syncRetryTimer)
      _syncRetryTimer = null
    }
  } catch(e) {
    _syncFailCount = (_syncFailCount || 0) + 1
    _syncDirty = true
    if (_syncFailCount <= CLOUD_SYNC_LOG_THRESHOLD) {
      console.warn(`[Storage] 云同步失败(${_syncFailCount}/${CLOUD_SYNC_MAX_FAIL}):`, e.message || e)
    }
    if (_syncFailCount >= CLOUD_SYNC_MAX_FAIL) {
      if (!_syncRetryTimer) {
        console.warn('[Storage] 云同步连续失败，进入低频重试模式（本地存档正常，不丢数据）')
        _syncRetryTimer = setInterval(() => {
          if (_syncDirty && !_syncing) {
            console.log('[Storage] 低频重试云同步...')
            _syncToCloud()
          }
        }, CLOUD_SYNC_RETRY_INTERVAL_MS)
      }
      _syncDisabled = true
    }
  } finally {
    _syncing = false
    if (_syncPending && !_syncDisabled) {
      _syncPending = false
      debounceSyncToCloud(_dataRef)
    }
  }
}
// ===== 防抖同步（每次 _save 后调用） =====
function debounceSyncToCloud(data) {
  _dataRef = data
  if (!_cloudInitDone) {
    _pendingSync = true
    return
  }
  if (_syncDisabled) return
  if (_cloudSyncTimer) clearTimeout(_cloudSyncTimer)
  const delay = _syncFailCount > 0
    ? Math.min(CLOUD_SYNC_BASE_DELAY_MS * Math.pow(2, _syncFailCount), CLOUD_SYNC_MAX_BACKOFF_MS)
    : CLOUD_SYNC_BASE_DELAY_MS
  _cloudSyncTimer = setTimeout(() => {
```

```
    _cloudSyncTimer = null
    _syncToCloud()
  }, delay)
}
// ===== 微信端辅助 =====
async function _ensureCollections() {
  const r = await P.cloud.callFunction({ name: 'initCollections' })
  if (r.result && r.result.errors && r.result.errors.length) {
    console.warn('创建集合异常:', r.result.errors)
  }
}
async function _getOpenid() {
  const r = await P.cloud.callFunction({ name: 'getOpenid' })
  _openid = (r.result && r.result.openid) || ''
}
// ===== 初始化入口 =====
async function init(persistData, opts) {
  _dataRef = persistData
  _localKey = opts.localKey
  _currentVersion = opts.currentVersion
  _runMigrations = opts.runMigrations
  if (P.isDouyin) {
    try {
      await api.login()
      _cloudReady = true
      console.log('[Storage] 抖音端 API 登录成功')
    } catch(e) {
      console.warn('[Storage] 抖音端 API 登录失败:', e.message || e)
      _cloudReady = true
      try {
        await api.getPlayerData()
        console.log('[Storage] 抖音端 API 直连测试通过')
      } catch(e2) {
        console.warn('[Storage] 抖音端后端不可达，本次会话云同步停用（本地存档正常）:', e2.message || e2)
        _cloudReady = false
      }
    }
  } else {
    try {
      P.cloud.init({ env: CLOUD_ENV, traceUser: true })
      _cloudReady = true
    } catch(e) {
      console.warn('Cloud init failed:', e)
      return
    }
    try { await _ensureCollections() } catch(e) {}
    try { await _getOpenid() } catch(e) { console.warn('Get openid failed:', e) }
  }
  if (_cloudReady && P.isWeChat && _openid) await _syncFromCloud()
  if (_cloudReady && P.isDouyin) await _syncFromCloud()
```

```
  _cloudInitDone = true
  if (_pendingSync && _cloudReady) {
    _pendingSync = false
    _syncToCloud()
  }
}
function isReady() { return _cloudReady }
function getOpenid() { return _openid }
module.exports = {
  init,
  syncToCloud: _syncToCloud,
  syncFromCloud: _syncFromCloud,
  debounceSyncToCloud,
  isReady,
  getOpenid,
}
/**
 * 全局数值常量集中管理 — 灵宠消消塔
 * 各域名前缀: STAMINA_*, CLOUD_*, RANK_*, TOWER_*, CULT_*, POOL_*, IDLE_*
 */
// ===== 体力系统 (STAMINA) =====
const STAMINA_RECOVER_INTERVAL_MS = 30 * 60 * 1000
const STAMINA_INITIAL = 100
const STAMINA_SIDEBAR_REWARD = 30
// ===== 云同步 (CLOUD) =====
const CLOUD_SYNC_BASE_DELAY_MS = 2000
const CLOUD_SYNC_MAX_BACKOFF_MS = 120000
const CLOUD_SYNC_MAX_FAIL = 5
const CLOUD_SYNC_LOG_THRESHOLD = 3
const CLOUD_SYNC_RETRY_INTERVAL_MS = 120000
// ===== 排行榜 (RANK) =====
const RANK_CACHE_TTL_MS = 30000
// ===== 塔层 / 肉鸽 (TOWER) =====
const TOWER_MAX_FLOOR = 30
const TOWER_COUNTER_MUL = 2.5
const TOWER_COUNTERED_MUL = 0.5
const TOWER_RECENT_LIMIT = 3
const TOWER_BASE_EVENT_WEIGHTS = {
  battle:    70,
  elite:     8,
  adventure:  5,
  shop:      4,
  rest:      3,
}
const TOWER_SHOP_DISPLAY_COUNT = 4
const TOWER_SHOP_FREE_COUNT = 1
const TOWER_SHOP_HP_COST_PCT = 15
// ===== 修炼系统 (CULT) =====
const CULT_MAX_LEVEL = 60
const CULT_EXP_BASE = 400
```

```
const CULT_EXP_LINEAR = 100
const CULT_EXP_POW_EXP = 1.6
const CULT_EXP_POW_COEFF = 6
const CULT_KILL_BOSS_BASE = 30
const CULT_KILL_BOSS_FLOOR_COEFF = 4
const CULT_KILL_ELITE_BASE = 15
const CULT_KILL_ELITE_FLOOR_COEFF = 3
const CULT_KILL_NORMAL_BASE = 5
const CULT_KILL_NORMAL_FLOOR_COEFF = 2
// ===== 灵宠池 (POOL) =====
const POOL_MAX_LV = 40
const POOL_ADV_MAX_LV = 60
const POOL_TIER_EXP_MUL = { T3: 1.0, T2: 1.3, T1: 1.6 }
const POOL_EXP_BASE = 20
const POOL_EXP_LINEAR = 8
const POOL_EXP_POW_EXP = 1.4
const POOL_EXP_POW_COEFF = 0.5
const POOL_STAR_FRAG_COST = { 2: 5, 3: 15, 4: 30 }
const POOL_STAR_LV_REQ = { 2: 10, 3: 20, 4: 45 }
const POOL_STAR_ATK_MUL = { 1: 1.0, 2: 1.3, 3: 1.7, 4: 2.2 }
const POOL_FRAGMENT_TO_EXP = 40
const POOL_ENTRY_LEVEL = 5
const POOL_ENTRY_FRAGMENTS = 2
const POOL_T3_LV_BONUS_RATE = 0.8
const POOL_ROGUE_EXP_RATIO = 0.3
const POOL_ROGUE_FLOOR_BONUS = 2
const POOL_ROGUE_CLEAR_BONUS = 200
// ===== 灵宠派遣 / 挂机 (IDLE) =====
const IDLE_MAX_SLOTS = 3
const IDLE_FRAG_INTERVAL = 4 * 3600 * 1000
const IDLE_MAX_ACCUMULATE = 24 * 3600 * 1000
const IDLE_PET_EXP_PER_HOUR = 8
const IDLE_PET_LV_EXP_FACTOR = 0.02
module.exports = {
  // STAMINA
  STAMINA_RECOVER_INTERVAL_MS,
  STAMINA_INITIAL,
  STAMINA_SIDEBAR_REWARD,
  // CLOUD
  CLOUD_SYNC_BASE_DELAY_MS,
  CLOUD_SYNC_MAX_BACKOFF_MS,
  CLOUD_SYNC_MAX_FAIL,
  CLOUD_SYNC_LOG_THRESHOLD,
  CLOUD_SYNC_RETRY_INTERVAL_MS,
  // RANK
  RANK_CACHE_TTL_MS,
  // TOWER
  TOWER_MAX_FLOOR,
  TOWER_COUNTER_MUL,
  TOWER_COUNTERED_MUL,
```

```
  TOWER_RECENT_LIMIT,
  TOWER_BASE_EVENT_WEIGHTS,
  TOWER_SHOP_DISPLAY_COUNT,
  TOWER_SHOP_FREE_COUNT,
  TOWER_SHOP_HP_COST_PCT,
  // CULT
  CULT_MAX_LEVEL,
  CULT_EXP_BASE,
  CULT_EXP_LINEAR,
  CULT_EXP_POW_EXP,
  CULT_EXP_POW_COEFF,
  CULT_KILL_BOSS_BASE,
  CULT_KILL_BOSS_FLOOR_COEFF,
  CULT_KILL_ELITE_BASE,
  CULT_KILL_ELITE_FLOOR_COEFF,
  CULT_KILL_NORMAL_BASE,
  CULT_KILL_NORMAL_FLOOR_COEFF,
  // POOL
  POOL_MAX_LV,
  POOL_ADV_MAX_LV,
  POOL_TIER_EXP_MUL,
  POOL_EXP_BASE,
  POOL_EXP_LINEAR,
  POOL_EXP_POW_EXP,
  POOL_EXP_POW_COEFF,
  POOL_STAR_FRAG_COST,
  POOL_STAR_LV_REQ,
  POOL_STAR_ATK_MUL,
  POOL_FRAGMENT_TO_EXP,
  POOL_ENTRY_LEVEL,
  POOL_ENTRY_FRAGMENTS,
  POOL_T3_LV_BONUS_RATE,
  POOL_ROGUE_EXP_RATIO,
  POOL_ROGUE_FLOOR_BONUS,
  POOL_ROGUE_CLEAR_BONUS,
  // IDLE
  IDLE_MAX_SLOTS,
  IDLE_FRAG_INTERVAL,
  IDLE_MAX_ACCUMULATE,
  IDLE_PET_EXP_PER_HOUR,
  IDLE_PET_LV_EXP_FACTOR,
}
/**
 * 修炼系统配置 — 等级 + 加点制
 *
 * 核心流程：打肉鸽/灵兽秘境 → 累积经验 → 升级 → 获得修炼点 → 分配到升级树
 * 所有修炼属性仅在灵兽秘境模式（battleMode === 'stage'）中生效
 */
const {
  CULT_MAX_LEVEL,
```

```
  CULT_EXP_BASE,
  CULT_EXP_LINEAR,
  CULT_EXP_POW_EXP,
  CULT_EXP_POW_COEFF,
  CULT_KILL_BOSS_BASE,
  CULT_KILL_BOSS_FLOOR_COEFF,
  CULT_KILL_ELITE_BASE,
  CULT_KILL_ELITE_FLOOR_COEFF,
  CULT_KILL_NORMAL_BASE,
  CULT_KILL_NORMAL_FLOOR_COEFF,
} = require('./constants')
// ===== 等级经验表 =====
// 总计 60 级，每级给 1 修炼点，58 点可把修炼树全部点满，多出 2 点作为容错
const MAX_LEVEL = CULT_MAX_LEVEL
function expToNextLevel(level) {
  if (level >= MAX_LEVEL) return Infinity
  return Math.floor(CULT_EXP_BASE + level * CULT_EXP_LINEAR + Math.pow(level, CULT_EXP_POW_EXP) *...
}
// 经验节奏（参考）：通天塔10层失败≈300 exp，约2~3局升一级
// Lv1→2: 506   Lv5→6: 978   Lv10→11: 1638   Lv20→21: 3123
// Lv30→31: 4786   Lv50→51: 8536   Lv59→60: 10388
// ===== 修炼树配置（加点制）=====
// 每项每级消耗 1 修炼点
const CULT_CONFIG = {
  body:    { name:'体魄', theme:'淬体', maxLv:20, perLv:5,    unit:'HP上限',   desc:'灵兽秘境中英雄HP上限' },
  spirit:  { name:'灵力', theme:'通脉', maxLv:15, perLv:1,    unit:'心珠回复', desc:'灵兽秘境中心珠回复基数' },
  wisdom:  { name:'悟性', theme:'感悟', maxLv:5,  perLv:0.15, unit:'s转珠时间', desc:'灵兽秘境中转珠操作时间' },
  defense: { name:'根骨', theme:'筑基', maxLv:10, perLv:2,    unit:'减伤',     desc:'灵兽秘境中每次受伤减免固定值' },
  sense:   { name:'神识', theme:'开窍', maxLv:8,  perLv:8,    unit:'护盾',     desc:'灵兽秘境中开局获得护盾' },
}
const CULT_KEYS = ['body', 'spirit', 'wisdom', 'defense', 'sense']
// 修炼树总共需要的点数：20+15+5+10+8 = 58
const TOTAL_POINTS_NEEDED = CULT_KEYS.reduce((s, k) => s + CULT_CONFIG[k].maxLv, 0)
/**
 * 当前等级的累计效果值
 */
function effectValue(key, level) {
  const cfg = CULT_CONFIG[key]
  if (!cfg || level <= 0) return 0
  return +(level * cfg.perLv).toFixed(2)
}
/**
 * 已分配的总点数
 */
function usedPoints(levels) {
  let sum = 0
  for (const k of CULT_KEYS) sum += (levels[k] || 0)
  return sum
}
// ===== 修炼境界（由等级决定）=====
```

```
1001 const REALMS = [
1002   { minLv: 0,  name: '凡人' },
1003   { minLv: 1,  name: '感气期' },
1004   { minLv: 5,  name: '练气期' },
1005   { minLv: 15, name: '筑基期' },
1006   { minLv: 30, name: '金丹期' },
1007   { minLv: 45, name: '元婴期' },
1008   { minLv: 58, name: '化神期' },
1009 ]
1010 function currentRealm(level) {
1011   let realm = REALMS[0]
1012   for (const r of REALMS) {
1013     if (level >= r.minLv) realm = r
1014     else break
1015   }
1016   return realm
1017 }
1018 function nextRealm(level) {
1019   for (const r of REALMS) {
1020     if (r.minLv > level) return r
1021   }
1022   return null
1023 }
1024 /**
1025  * 击杀经验公式（统一数值，battle.js / touchHandlers.js 共用）
1026  * @param {object} enemy - 包含 isBoss, isElite 标记
1027  * @param {number} floor - 当前层数
1028  */
1029 function killExpBase(enemy, floor) {
1030   if (!enemy) return 0
1031   if (enemy.isBoss) return CULT_KILL_BOSS_BASE + floor * CULT_KILL_BOSS_FLOOR_COEFF
1032   if (enemy.isElite) return CULT_KILL_ELITE_BASE + floor * CULT_KILL_ELITE_FLOOR_COEFF
1033   return CULT_KILL_NORMAL_BASE + floor * CULT_KILL_NORMAL_FLOOR_COEFF
1034 }
1035 module.exports = {
1036   MAX_LEVEL, expToNextLevel,
1037   CULT_CONFIG, CULT_KEYS, TOTAL_POINTS_NEEDED,
1038   effectValue, usedPoints,
1039   REALMS, currentRealm, nextRealm,
1040   killExpBase,
1041 }
1042 /**
1043  * 灵宠池配置 — 独立培养体系
1044  *
1045  * 核心设计:
1046  * - 共享经验池：肉鸽和固定关卡都产出宠物经验，玩家自主分配
1047  * - 按档位差异化升级成本：T3便宜易练，T1昂贵但强
1048  * - 碎片分解：不需要的碎片可转化为宠物经验
1049  * - 独立于肉鸽内★1→★3的简化升星，这里是长期局外养成
1050  */
```

```
const { getPetTier, getPetById } = require('./pets')
const {
  POOL_MAX_LV,
  POOL_ADV_MAX_LV,
  POOL_TIER_EXP_MUL,
  POOL_EXP_BASE,
  POOL_EXP_LINEAR,
  POOL_EXP_POW_EXP,
  POOL_EXP_POW_COEFF,
  POOL_STAR_FRAG_COST,
  POOL_STAR_LV_REQ,
  POOL_STAR_ATK_MUL,
  POOL_FRAGMENT_TO_EXP,
  POOL_ENTRY_LEVEL,
  POOL_ENTRY_FRAGMENTS,
  POOL_T3_LV_BONUS_RATE,
  POOL_ROGUE_EXP_RATIO,
  POOL_ROGUE_FLOOR_BONUS,
  POOL_ROGUE_CLEAR_BONUS,
  IDLE_MAX_SLOTS,
  IDLE_FRAG_INTERVAL,
  IDLE_MAX_ACCUMULATE,
  IDLE_PET_EXP_PER_HOUR,
  IDLE_PET_LV_EXP_FACTOR,
} = require('./constants')
// ===== 等级系统 =====
const TIER_EXP_MUL = POOL_TIER_EXP_MUL
/**
 * 灵宠升级所需经验（受档位影响）
 * @param {number} level - 当前等级
 * @param {string} tier - 宠物档位 'T1'|'T2'|'T3'
 */
function petExpToNextLevel(level, tier) {
  const base = Math.floor(POOL_EXP_BASE + level * POOL_EXP_LINEAR + Math.pow(level, POOL_EXP_POW_...
  const mul = TIER_EXP_MUL[tier] || 1.0
  return Math.floor(base * mul)
}
// ===== 星级系统 =====
const FRAGMENT_TO_EXP = POOL_FRAGMENT_TO_EXP
// ===== 攻击力计算 =====
/**
 * 灵宠池中宠物的攻击力（仅用于固定关卡）
 * @param {object} poolPet - 灵宠池中的宠物记录 { id, level, star, ... }
 */
function getPoolPetAtk(poolPet) {
  const basePet = getPetById(poolPet.id)
  if (!basePet) return 0
  const baseAtk = basePet.atk
  const tier = getPetTier(poolPet.id)
  const lvBonus = tier === 'T3' ? Math.floor(poolPet.level * POOL_T3_LV_BONUS_RATE) : poolPet.level
```

```
  const starMul = POOL_STAR_ATK_MUL[poolPet.star] || 1.0
  return Math.floor((baseAtk + lvBonus) * starMul)
}
// ===== 入池默认值 =====
const ENTRY_LEVEL = POOL_ENTRY_LEVEL
const ENTRY_FRAGMENTS = POOL_ENTRY_FRAGMENTS
// ===== 肉鸽局结算宠物经验 =====
/**
 * 肉鸽结算宠物经验公式
 * 独立于修炼经验，按消除/连击/击杀/层数计算
 * @param {object} expDetail - { elimExp, comboExp, killExp }
 * @param {number} floor - 最终到达层数
 * @param {boolean} cleared - 是否通关
 */
function calcRoguelikePetExp(expDetail, floor, cleared) {
  const rawExp = (expDetail.elimExp || 0) + (expDetail.comboExp || 0) + (expDetail.killExp || 0)
  const baseFromCombat = Math.floor(rawExp * POOL_ROGUE_EXP_RATIO)
  const floorBonus = floor * POOL_ROGUE_FLOOR_BONUS
  const clearBonus = cleared ? POOL_ROGUE_CLEAR_BONUS : 0
  return baseFromCombat + floorBonus + clearBonus
}
// ===== 灵宠派遣（挂机）系统 =====
// 派遣常量从 constants.js 导入
/**
 * 计算派遣产出
 * @param {number} elapsedMs - 经过时间（毫秒）
 * @param {number} petLevel  - 宠物等级（影响经验产出）
 * @returns {{ fragments: number, petExp: number }}
 */
function calcIdleReward(elapsedMs, petLevel) {
  const capped = Math.min(elapsedMs, IDLE_MAX_ACCUMULATE)
  const fragments = Math.floor(capped / IDLE_FRAG_INTERVAL)
  const hours = capped / (3600 * 1000)
  const petExp = Math.floor(hours * IDLE_PET_EXP_PER_HOUR * (1 + petLevel * IDLE_PET_LV_EXP_FACTOR))
  return { fragments, petExp }
}
module.exports = {
  POOL_MAX_LV,
  POOL_ADV_MAX_LV,
  TIER_EXP_MUL,
  petExpToNextLevel,
  POOL_STAR_FRAG_COST,
  POOL_STAR_LV_REQ,
  POOL_STAR_ATK_MUL,
  FRAGMENT_TO_EXP,
  getPoolPetAtk,
  ENTRY_LEVEL,
  ENTRY_FRAGMENTS,
  calcRoguelikePetExp,
  IDLE_MAX_SLOTS,
```

```
  IDLE_FRAG_INTERVAL,
  IDLE_MAX_ACCUMULATE,
  IDLE_PET_EXP_PER_HOUR,
  calcIdleReward,
}
/**
 * 宠物系统 — 灵宠消消塔（爽感重做版）
 * 5属性 × 20只 = 100只宠物
 * 核心改动：CD大幅缩短(3-6)、效果大幅增强、变珠整行/列级别
 * 新增"五行共鸣"必杀技机制（4只宠物技能全部就绪时可触发）
 */
// ===== 技能效果类型 =====
// dmgBoost  — 下次该属性伤害×N倍（改为倍率制，体感更强）
// convertBead — 随机N颗珠子变为指定属性（数量大幅提升）
// convertRow — 整行变为指定属性（新增）
// convertCol — 整列变为指定属性（新增）
// convertCross — 十字变为指定属性（新增）
// shield    — 获得护盾(N点)
// reduceDmg — 本回合受到伤害-X%
// stun      — 敌人眩晕N回合（跳过攻击）
// comboPlus — 本次Combo+N（虚拟额外combo段）
// extraTime — 转珠时间+N秒
// ignoreDefPct — 下次该属性攻击无视X%防御
// revive    — 抵挡一次致死伤害（本局1次）
// healPct   — 立即回复X%最大血量
// healFlat  — 立即回复N点血量
// dot       — 敌人中毒/灼烧，每回合N点，持续M回合
// instantDmg — 立即造成一次该属性伤害（倍率pct%基础攻击）
// hpMaxUp   — 血量上限+X%
// heartBoost — 心珠效果+X%（或翻倍1回合）
// allDmgUp  — 全队所有属性伤害+X%，持续N回合
// allAtkUp  — 全队攻击+X%，持续N回合
// allDefUp  — 全队防御+X%，持续N回合
// critBoost — 暴击率+X%，持续N回合
// critDmgUp — 暴击伤害+X%
// reflectPct — 反弹X%伤害，持续N回合
// immuneCtrl — 免疫控制N回合
// beadRateUp — 指定属性珠出现概率大幅提升1回合
// comboNeverBreak — 本次Combo不会断
// healOnElim — 消除该属性珠时回血X%
// shieldOnElim — 消除该属性珠时获得护盾N点
// lowHpDmgUp — 残血时临时伤害+X%
// stunPlusDmg — 眩晕+下次伤害×N倍
// fullHeal  — 满血回复
// allHpMaxUp — 全队血量上限+X%
// dmgImmune — 本回合受到伤害变为1点
// guaranteeCrit — 本次攻击必定暴击+暴击伤害提升
// comboDmgUp — Combo伤害额外+X%
// percentAtkUp — 每N层全队攻击+X%（永久叠加）
// onKillHeal — 击杀怪物后回血X%
```

```
// multiHit — 连续攻击N次（新增，每次造成X%伤害）
// explode  — 消除后引发爆炸，对敌人造成额外伤害（新增）
// replaceBeads — 全场指定属性珠全部转为另一属性（五行克制转珠）
const PETS = {
 // ===== （一）金属性宠物（20只） =====
 metal: [
  { id:'m1',  name:'金锋灵猫',  atk:8,  skill:{ name:'锋芒斩', desc:'下次金属性伤害×2倍', type:'dmgBoost', att...
  { id:'m2',  name:'锐金鼠将',  atk:9,  skill:{ name:'金珠阵', desc:'整行珠子变为金珠', type:'convertRow', att...
  { id:'m3',  name:'玄甲金狮',  atk:9,  skill:{ name:'金甲术', desc:'获得护盾12点+本回合免伤25%', type:'shieldPl...
  { id:'m4',  name:'天罡金鹏',  atk:10, skill:{ name:'天罡击', desc:'对敌人造成250%攻击力伤害', type:'instantDmg...
  { id:'m5',  name:'碎金战将',  atk:11, skill:{ name:'金缚锁', desc:'敌人眩晕2回合', type:'stun', dur:2 }, c...
  { id:'m6',  name:'金光剑灵',  atk:11, skill:{ name:'剑气纵横', desc:'Combo+2，且本次Combo不会断', type:'comb...
  { id:'m7',  name:'金罡守卫',  atk:9,  skill:{ name:'罡气化愈', desc:'全部木珠转为心珠', type:'replaceBeads', ...
  { id:'m8',  name:'鸣金神雀',  atk:10, skill:{ name:'鸣金诀', desc:'转珠时间+3秒+金珠概率大增', type:'extraTimeP...
  { id:'m9',  name:'破甲金将',  atk:12, skill:{ name:'破甲斩', desc:'下次金攻无视全部防御+伤害×1.5', type:'ignoreD...
  { id:'m10', name:'九天金凰',  atk:12, skill:{ name:'金身不灭', desc:'抵挡一次致死伤害+回复50%血量', type:'reviveP...
  { id:'m11', name:'锐金斥候',  atk:11, skill:{ name:'锐金三连', desc:'连续攻击3次，每次100%攻击力', type:'multiHi...
  { id:'m12', name:'金纹战将',  atk:11, skill:{ name:'金纹化木', desc:'全部木珠转金珠+金伤+30%', type:'replaceBe...
  { id:'m13', name:'金影刺客',  atk:12, skill:{ name:'致命金刺', desc:'必定暴击+暴击伤害+50%', type:'guaranteeC...
  { id:'m14', name:'金甲神卫',  atk:10, skill:{ name:'金甲战吼', desc:'全队攻击+35%持续2回合', type:'allAtkUp',...
  { id:'m15', name:'金虹使者',  atk:12, skill:{ name:'金虹贯日', desc:'十字形珠子全变金珠', type:'convertCross',...
  { id:'m16', name:'金罡战魂',  atk:11, skill:{ name:'战魂爆发', desc:'全队攻击+50%持续2回合', type:'allAtkUp',...
  { id:'m17', name:'金翎神使',  atk:11, skill:{ name:'金翎风暴', desc:'随机8颗变金珠+金珠概率大增', type:'convertBe...
  { id:'m18', name:'金锋战神',  atk:13, skill:{ name:'战神降临', desc:'全队攻击+40%持续3回合+必暴击1回合', type:'war...
  { id:'m19', name:'金耀星君',  atk:12, skill:{ name:'星耀裂空', desc:'眩晕2回合+下次金伤×3倍', type:'stunPlusDm...
  { id:'m20', name:'万钧金神',  atk:14, skill:{ name:'万钧神威', desc:'立即造成500%金属性爆裂伤害', type:'instantD...
 ],
 // ===== （二）木属性宠物（20只） =====
 wood: [
  { id:'w1',  name:'青灵木鹿',  atk:8,  skill:{ name:'春回大地', desc:'立即回复30%血量', type:'healPct', pct:...
  { id:'w2',  name:'藤萝灵蛇',  atk:8,  skill:{ name:'剧毒蛇牙', desc:'剧毒：每回合25点，持续3回合', type:'dot', dm...
  { id:'w3',  name:'苍木灵熊',  atk:9,  skill:{ name:'熊力化愈', desc:'全部土珠转为心珠', type:'replaceBeads', ...
  { id:'w4',  name:'万木灵狐',  atk:9,  skill:{ name:'万木化珠', desc:'整列珠子变木珠', type:'convertCol', att...
  { id:'w5',  name:'灵木仙子',  atk:10, skill:{ name:'仙子化土', desc:'全部土珠转木珠+每回合回血8点2回合', type:'repla...
  { id:'w6',  name:'青木战灵',  atk:11, skill:{ name:'木灵爆发', desc:'下次木属性伤害×2.5倍', type:'dmgBoost', ...
  { id:'w7',  name:'缠枝藤君',  atk:10, skill:{ name:'缠枝锁魂', desc:'敌人眩晕2回合+中毒18/回合', type:'stunDot'...
  { id:'w8',  name:'枯木老妖',  atk:9,  skill:{ name:'枯木缠魂', desc:'眩晕1回合+中毒20/回合持续3回合', type:'stunD...
  { id:'w9',  name:'木灵使者',  atk:11, skill:{ name:'灵木滋生', desc:'下回合木珠出现概率大增', type:'beadRateUp',...
  { id:'w10', name:'万木之主',  atk:12, skill:{ name:'万木天威', desc:'全队全属性伤害+40%持续3回合', type:'allDmgU...
  { id:'w11', name:'青藤守卫',  atk:9,  skill:{ name:'藤甲壁垒', desc:'护盾30点+减伤20%持续2回合', type:'shieldP...
  { id:'w12', name:'翠竹灵蟋',  atk:11, skill:{ name:'翠竹连击', desc:'连续攻击3次，每次120%木属性伤害', type:'multi...
  { id:'w13', name:'灵芝仙菇',  atk:9,  skill:{ name:'净化万毒', desc:'清除所有负面+免疫控制2回合', type:'purify', ...
  { id:'w14', name:'苍蟒木蛟',  atk:11, skill:{ name:'蛟龙木爆', desc:'下次木属性伤害×2倍', type:'dmgBoost', at...
  { id:'w15', name:'木灵仙鹿',  atk:10, skill:{ name:'仙鹿化珠', desc:'全部土珠转木珠+全队攻击+20%', type:'replace...
  { id:'w16', name:'千年古藤',  atk:12, skill:{ name:'古藤缠天', desc:'十字形珠子全变木珠', type:'convertCross',...
  { id:'w17', name:'碧玉螳螂',  atk:11, skill:{ name:'螳螂无双', desc:'Combo不断+Combo伤害+50%', type:'comb...
  { id:'w18', name:'青鸾翠雀',  atk:11, skill:{ name:'翠雀风舞', desc:'整行变木珠+木珠概率大增', type:'convertRow'...
  { id:'w19', name:'万木神龟',  atk:12, skill:{ name:'神龟之力', desc:'全队防御+40%持续3回合', type:'allDefUp',...
  { id:'w20', name:'神木麒麟',  atk:14, skill:{ name:'神木灭世', desc:'立即500%木属性爆裂伤害', type:'instantDmg...
 ],
```

```
// ===== （三）水属性宠物（20只） =====
water: [
  { id:'s1',  name:'沧澜水雀',  atk:8,  skill:{ name:'水珠涌现', desc:'整列珠子变水珠', type:'convertCol', att...
  { id:'s2',  name:'冰魄灵龟',  atk:9,  skill:{ name:'冰魄化愈', desc:'全部火珠转为心珠', type:'replaceBeads', ...
  { id:'s3',  name:'海灵蛟童',  atk:9,  skill:{ name:'寒冰封印', desc:'冰冻敌人2回合', type:'stun', dur:2 }, ...
  { id:'s4',  name:'玄水蛟龙',  atk:10, skill:{ name:'蛟龙怒击', desc:'250％水属性直接伤害', type:'instantDmg',...
  { id:'s5',  name:'碧波灵蛙',  atk:9,  skill:{ name:'碧波愈泉', desc:'回复40％血量', type:'healPct', pct:40...
  { id:'s6',  name:'流水灵鱼',  atk:11, skill:{ name:'水灵爆涌', desc:'下次水属性伤害×2.5倍', type:'dmgBoost', ...
  { id:'s7',  name:'寒冰灵蟹',  atk:10, skill:{ name:'寒冰之壁', desc:'反弹25％伤害持续2回合', type:'reflectPct'...
  { id:'s8',  name:'海魂巨鲸',  atk:11, skill:{ name:'鲸吞怒涛', desc:'全属性伤害+30％持续2回合', type:'allDmgUp'...
  { id:'s9',  name:'凝水灵蚌',  atk:9,  skill:{ name:'凝水护体', desc:'免疫所有控制2回合', type:'immuneCtrl', d...
  { id:'s10', name:'沧海龙神',  atk:13, skill:{ name:'龙神覆海', desc:'全场一半珠子变水珠（约18颗）', type:'convertB...
  { id:'s11', name:'冰玄灵蛾',  atk:11, skill:{ name:'冰玄风暴', desc:'随机8颗变水珠+水珠概率大增', type:'convertBe...
  { id:'s12', name:'沧澜海蛇',  atk:12, skill:{ name:'海蛇三连', desc:'连续攻击3次，每次120％水伤害', type:'multiHi...
  { id:'s13', name:'玄水灵蟾',  atk:10, skill:{ name:'灵蟾冰封', desc:'冰冻敌人1回合+水伤×2倍', type:'stunPlusDm...
  { id:'s14', name:'冰魄灵鹤',  atk:12, skill:{ name:'冰魄封天', desc:'眩晕2回合+水伤×2倍', type:'stunPlusDmg'...
  { id:'s15', name:'海灵水母',  atk:11, skill:{ name:'水母幻术', desc:'转珠时间+3秒+Combo不断', type:'extraTim...
  { id:'s16', name:'水镜灵蝶',  atk:11, skill:{ name:'水镜反射', desc:'反弹30％伤害持续2回合', type:'reflectPct'...
  { id:'s17', name:'沧澜鲲鹏',  atk:14, skill:{ name:'鲲鹏怒涛', desc:'500％水属性爆裂伤害', type:'instantDmg',...
  { id:'s18', name:'玄水神蛟',  atk:12, skill:{ name:'神蛟护佑', desc:'全队防御+50％持续3回合', type:'allDefUp',...
  { id:'s19', name:'水纹灵獭',  atk:11, skill:{ name:'灵獭化火', desc:'全部火珠转水珠+水珠概率大增', type:'replaceBe...
  { id:'s20', name:'冰凰神鸟',  atk:11, skill:{ name:'冰凰绝对', desc:'免疫所有控制2回合+护盾20点', type:'immuneSh...
],
// ===== （四）火属性宠物（20只） =====
fire: [
  { id:'f1',  name:'赤焰火狐',  atk:8,  skill:{ name:'焰爪连击', desc:'下次火属性伤害×2倍', type:'dmgBoost', at...
  { id:'f2',  name:'焚天火狼',  atk:9,  skill:{ name:'烈火珠阵', desc:'整行珠子变火珠', type:'convertRow', att...
  { id:'f3',  name:'烈阳火凰',  atk:11, skill:{ name:'凤凰烈焰', desc:'350％火属性直接伤害', type:'instantDmg',...
  { id:'f4',  name:'炎狱火麟',  atk:11, skill:{ name:'炎狱暴击', desc:'必暴击+暴击伤害+50％', type:'guaranteeCr...
  { id:'f5',  name:'爆炎火蟾',  atk:10, skill:{ name:'烈焰灼烧', desc:'灼烧：每回合30点持续3回合', type:'dot', dmg...
  { id:'f6',  name:'火莲灵花',  atk:11, skill:{ name:'火莲绽放', desc:'Combo伤害额外+60％', type:'comboDmgUp...
  { id:'f7',  name:'焚天火鸦',  atk:11, skill:{ name:'焚天之怒', desc:'敌人眩晕2回合', type:'stun', dur:2 }, ...
  { id:'f8',  name:'赤炎火蝎',  atk:11, skill:{ name:'炎魔连击', desc:'Combo+4且不会断', type:'comboPlusNev...
  { id:'f9',  name:'火灵赤蛇',  atk:10, skill:{ name:'蛇焰化愈', desc:'全部金珠转为心珠', type:'replaceBeads', ...
  { id:'f10', name:'朱雀神火',  atk:13, skill:{ name:'朱雀圣焰', desc:'全队暴击率+50％持续2回合', type:'critBoost...
  { id:'f11', name:'焚天火猿',  atk:12, skill:{ name:'焚魂珠阵', desc:'随机8颗变火珠+火珠概率大增', type:'convertBe...
  { id:'f12', name:'炎狱火蜥',  atk:12, skill:{ name:'炎狱三连', desc:'连续攻击3次，每次130％火伤害', type:'multiHi...
  { id:'f13', name:'烈阳火鹰',  atk:11, skill:{ name:'烈阳风暴', desc:'全队主动攻击，每只150％伤害', type:'teamAtta...
  { id:'f14', name:'火凰灵蝶',  atk:12, skill:{ name:'凰翼爆炎', desc:'暴击伤害+40％+必暴击1回合', type:'critDmgU...
  { id:'f15', name:'炎爆火鼠',  atk:12, skill:{ name:'炎爆裂天', desc:'眩晕2回合+火伤×3倍', type:'stunPlusDmg'...
  { id:'f16', name:'焚天火蟒',  atk:14, skill:{ name:'天火焚灭', desc:'500％火属性爆裂伤害+灼烧', type:'instantDm...
  { id:'f17', name:'赤焰麒麟',  atk:13, skill:{ name:'麒麟战焰', desc:'全队攻击+50％持续3回合', type:'allAtkUp',...
  { id:'f18', name:'火元灵龟',  atk:12, skill:{ name:'元火暴击', desc:'每段Combo暴击率+10％', type:'critBoost...
  { id:'f19', name:'炎狱火龙',  atk:12, skill:{ name:'龙炎破甲', desc:'火伤无视全部防御+伤害×1.5', type:'ignoreDe...
  { id:'f20', name:'火灵神猫',  atk:11, skill:{ name:'余烬爆发', desc:'残血时伤害+80％', type:'lowHpDmgUp', p...
],
// ===== （五）土属性宠物（20只） =====
earth: [
  { id:'e1',  name:'厚土石灵',  atk:9,  skill:{ name:'厚土化愈', desc:'全部水珠转为心珠', type:'replaceBeads', ...
  { id:'e2',  name:'山岳石怪',  atk:9,  skill:{ name:'山岳珠阵', desc:'整行珠子变土珠', type:'convertRow', att...
```

```
    { id:'e3',  name:'镇地石犀',  atk:10, skill:{ name:'镇地壁垒', desc:'获得护盾35点', type:'shield', val:35 ...
    { id:'e4',  name:'玄武圣兽',  atk:11, skill:{ name:'玄武震慑', desc:'敌人眩晕2回合', type:'stun', dur:2 }, ...
    { id:'e5',  name:'裂地穿山甲',  atk:12, skill:{ name:'裂地重击', desc:'下次土属性伤害×2.5倍', type:'dmgBoost',...
    { id:'e6',  name:'山岩石蟹',  atk:10, skill:{ name:'岩甲回春', desc:'血量上限+30%+立即回满', type:'hpMaxUp', ...
    { id:'e7',  name:'镇山石狮',  atk:11, skill:{ name:'石狮无畏', desc:'免疫所有控制2回合', type:'immuneCtrl', d...
    { id:'e8',  name:'大地灵鼹',  atk:11, skill:{ name:'大地反噬', desc:'反弹30%伤害2回合', type:'reflectPct', ...
    { id:'e9',  name:'玄土石蟒',  atk:12, skill:{ name:'蟒击碎岩', desc:'350%土属性直接伤害', type:'instantDmg',...
    { id:'e10', name:'后土神兽',  atk:13, skill:{ name:'后土庇佑', desc:'全队防御+60%持续3回合', type:'allDefUp',...
    { id:'e11', name:'厚土灵虫',  atk:11, skill:{ name:'土魂涌现', desc:'随机8颗变土珠+土珠概率大增', type:'convertBe...
    { id:'e12', name:'山岳灵兔',  atk:12, skill:{ name:'兔踢三连', desc:'连续攻击3次，每次120%土伤害', type:'multiHi...
    { id:'e13', name:'镇地石龙',  atk:11, skill:{ name:'镇地龙吼', desc:'全队防御+40%持续2回合', type:'allDefUp',...
    { id:'e14', name:'玄土灵蛤',  atk:12, skill:{ name:'蛤蟆震地', desc:'眩晕1回合+破甲（防御归零）', type:'stunBreak...
    { id:'e15', name:'裂地灵蚁',  atk:11, skill:{ name:'灵蚁化水', desc:'全部水珠转土珠+土珠概率大增', type:'replaceBe...
    { id:'e16', name:'山岩石象',  atk:11, skill:{ name:'象踏碎甲', desc:'眩晕1回合+破甲（防御减半）', type:'stunBreak...
    { id:'e17', name:'后土灵蚕',  atk:10, skill:{ name:'地脉转珠', desc:'十字形珠子全变土珠', type:'convertCross',...
    { id:'e18', name:'镇地神牛',  atk:14, skill:{ name:'神牛碎天', desc:'500%土属性爆裂伤害', type:'instantDmg',...
    { id:'e19', name:'厚土灵龟',  atk:11, skill:{ name:'灵龟镇水', desc:'全部水珠转土珠+全队防御+20%', type:'replace...
    { id:'e20', name:'玄武神君',  atk:13, skill:{ name:'玄武破天', desc:'眩晕2回合+土伤×3倍', type:'stunPlusDmg'...
  ],
}
// ===== 宠物强度分档（T1神兽 / T2灵兽 / T3幼兽） =====
// T1: ATK≥13 或拥有终极技能（500%爆裂、战神、复活、眩晕+3倍等）
// T2: ATK 10-12 且技能中等偏强
// T3: ATK 8-9 或技能偏弱/偏辅助
const PET_TIER = {
  T1: [
    'm10','m18','m19','m20',       // 九天金凰、金锋战神、金耀星君、万钧金神
    'w10','w20',                   // 万木之主、神木麒麟
    's10','s17',                   // 沧海龙神、沧澜鲲鹏
    'f10','f16','f17',             // 朱雀神火、焚天火蟒、赤焰麒麟
    'e10','e18','e20',             // 后土神兽、镇地神牛、玄武神君
  ],
  T2: [
    'm4','m5','m6','m9','m11','m13','m14','m15','m16','m17',  // 金属性中坚
    'w5','w6','w7','w12','w14','w15','w16','w17','w18','w19', // 木属性中坚
    's4','s6','s8','s11','s12','s14','s15','s16','s18','s19','s20', // 水属性中坚
    'f3','f4','f5','f6','f7','f8','f11','f12','f13','f14','f15','f18','f19','f20', // 火属性中坚
    'e4','e5','e8','e9','e11','e12','e13','e14','e15','e16','e17','e19', // 土属性中坚
  ],
  T3: [
    'm1','m2','m3','m7','m8','m12',  // 金属性幼兽
    'w1','w2','w3','w4','w8','w9','w11','w13', // 木属性幼兽
    's1','s2','s3','s5','s7','s9','s13', // 水属性幼兽
    'f1','f2','f9',                // 火属性幼兽
    'e1','e2','e3','e6','e7',      // 土属性幼兽
  ],
}
// 根据宠物ID获取档位
function getPetTier(id) {
  if (PET_TIER.T1.includes(id)) return 'T1'
```

```
  if (PET_TIER.T2.includes(id)) return 'T2'
  return 'T3'
}
// 不同获取渠道的档位权重
const TIER_WEIGHTS = {
  starter:  { T3:100, T2:0,  T1:0  },  // 开局初始：全T3
  normal:   { T3:50,  T2:40, T1:10 },  // 普通战斗
  elite:    { T3:15,  T2:55, T1:30 },  // 精英战斗
  boss:     { T3:0,   T2:40, T1:60 },  // BOSS战斗
  shop:     { T3:30,  T2:50, T1:20 },  // 商店招募
  adventure: { T3:0,   T2:30, T1:70 },  // 奇遇
}
// ===== ★3满星技能强化覆写 =====
// 每只宠物升到★3后技能质变，desc更新+数值/效果增强
const STAR3_SKILL_OVERRIDE = {
  // --- 金属性 ---
  m1:  { desc:'下次金属性伤害×3倍，持续2回合', pct:200, dur:2 },
  m2:  { desc:'整行变金珠+额外随机3颗变金珠', extra:3 },
  m3:  { desc:'护盾20点+本回合免伤35%+反弹15%伤害', val:20, reducePct:35, reflectPct:15 },
  m4:  { desc:'对敌造成350%伤害+无视50%防御', pct:350, ignoreDefPct:50 },
  m5:  { desc:'眩晕2回合+期间受伤+30%', dur:2, extraDmgPct:30 },
  m6:  { desc:'Combo+3不断+Combo伤害+25%', count:3, comboDmgPct:25 },
  m7:  { desc:'全部木珠转心珠+心珠回复+40%', toAttr:'heart', heartBoost:40 },
  m8:  { desc:'转珠+5秒+金珠概率大增+Combo+1', sec:5, bonusCombo:1 },
  m9:  { desc:'金攻无视全防+伤害×2倍', dmgMul:100 },
  m10: { desc:'抵挡致死+回复80%血量+无敌1回合', healPct:80, immuneDur:1 },
  m11: { desc:'连续攻击4次，每次120%攻击力', hits:4, pct:120 },
  m12: { desc:'全部木珠转金珠+金伤+50%', dmgBoost:50 },
  m13: { desc:'必暴击+暴击伤害+80%', critDmgBonus:80 },
  m14: { desc:'全队攻击+45%持续3回合', pct:45, dur:3 },
  m15: { desc:'十字形全变金珠+金属性伤害+40%', dmgBoost:40 },
  m16: { desc:'全队攻击+65%持续3回合', pct:65, dur:3 },
  m17: { desc:'随机12颗变金珠+金珠概率大增', count:12 },
  m18: { desc:'全队攻击+55%持续4回合+必暴击2回合', pct:55, dur:4, critDur:2 },
  m19: { desc:'眩晕3回合+下次金伤×4倍', stunDur:3, pct:300 },
  m20: { desc:'700%金属性爆裂伤害+溅射全体30%', pct:700, splash:30 },
  // --- 木属性 ---
  w1:  { desc:'回复50%血量+清除1个负面', pct:50, cleanse:1 },
  w2:  { desc:'剧毒40点/回合，持续4回合', dmg:40, dur:4 },
  w3:  { desc:'全部土珠转心珠+心珠回复+40%', heartBoost:40 },
  w4:  { desc:'整列变木珠+额外随机3颗变木珠', extra:3 },
  w5:  { desc:'全部土珠转木珠+回血12点/回合持续3回合', regen:12, regenDur:3 },
  w6:  { desc:'下次木属性伤害×3.5倍', pct:250 },
  w7:  { desc:'眩晕3回合+中毒30/回合持续3回合', dur:3, dotDmg:30 },
  w8:  { desc:'眩晕2回合+中毒30/回合持续4回合', dur:2, dotDmg:30, dotDur:4 },
  w9:  { desc:'木珠概率大增持续2回合', dur:2 },
  w10: { desc:'全队全属性伤害+55%持续4回合', pct:55, dur:4 },
  w11: { desc:'护盾45点+减伤30%持续3回合', val:45, reducePct:30, dur:3 },
  w12: { desc:'连续攻击4次，每次140%木伤害', hits:4, pct:140 },
  w13: { desc:'清除所有负面+免疫控制3回合+回复15%血量', immuneDur:3, healPct:15 },
```

```
w14: { desc:'下次木属性伤害×3倍', pct:200 },
w15: { desc:'全部土珠转木珠+全队攻击+30%', atkBoost:30 },
w16: { desc:'十字形全变木珠+木属性伤害+40%', dmgBoost:40 },
w17: { desc:'Combo不断+Combo伤害+70%', comboDmgPct:70 },
w18: { desc:'整行变木珠+木珠概率大增+木伤+20%', dmgBoost:20 },
w19: { desc:'全队防御+55%持续4回合', pct:55, dur:4 },
w20: { desc:'700%木属性爆裂伤害+全体回复15%', pct:700, teamHealPct:15 },
// --- 水属性 ---
s1: { desc:'整列变水珠+额外随机3颗变水珠', extra:3 },
s2: { desc:'全部火珠转心珠+心珠回复+40%', heartBoost:40 },
s3: { desc:'冰冻敌人2回合+受伤+25%', dur:2, extraDmgPct:25 },
s4: { desc:'350%水属性伤害+无视50%防御', pct:350, ignoreDefPct:50 },
s5: { desc:'回复60%血量+清除1个负面', pct:60, cleanse:1 },
s6: { desc:'下次水属性伤害×3.5倍', pct:250 },
s7: { desc:'反弹40%伤害持续3回合', pct:40, dur:3 },
s8: { desc:'全属性伤害+45%持续3回合', pct:45, dur:3 },
s9: { desc:'免疫所有控制3回合', dur:3 },
s10: { desc:'全场一半珠子变水珠+水伤+30%', count:18, dmgBoost:30 },
s11: { desc:'随机12颗变水珠+水珠概率大增', count:12 },
s12: { desc:'连续攻击4次，每次140%水伤害', hits:4, pct:140 },
s13: { desc:'冰冻2回合+水伤×3倍', stunDur:2, pct:200 },
s14: { desc:'眩晕3回合+水伤×3倍', stunDur:3, pct:200 },
s15: { desc:'转珠+5秒+Combo不断+Combo+1', sec:5, bonusCombo:1 },
s16: { desc:'反弹40%伤害持续3回合', pct:40, dur:3 },
s17: { desc:'700%水属性爆裂伤害+冰冻1回合', pct:700, stunDur:1 },
s18: { desc:'全队防御+65%持续4回合', pct:65, dur:4 },
s19: { desc:'全部火珠转水珠+水珠概率大增+水伤+25%', dmgBoost:25 },
s20: { desc:'免疫控制3回合+护盾35点', immuneDur:3, shieldVal:35 },
// --- 火属性 ---
f1: { desc:'下次火属性伤害×3倍，持续2回合', pct:200, dur:2 },
f2: { desc:'整行变火珠+额外随机3颗变火珠', extra:3 },
f3: { desc:'500%火属性直接伤害', pct:500 },
f4: { desc:'必暴击+暴击伤害+80%', critDmgBonus:80 },
f5: { desc:'灼烧40点/回合，持续4回合', dmg:40, dur:4 },
f6: { desc:'Combo伤害额外+80%', pct:80 },
f7: { desc:'眩晕2回合+期间受伤+30%', dur:2, extraDmgPct:30 },
f8: { desc:'Combo+6不断+Combo伤害+25%', count:6, comboDmgPct:25 },
f9: { desc:'全部金珠转心珠+心珠回复+40%', heartBoost:40 },
f10: { desc:'全队暴击率+70%持续3回合', pct:70, dur:3 },
f11: { desc:'随机12颗变火珠+火珠概率大增', count:12 },
f12: { desc:'连续攻击4次，每次150%火伤害', hits:4, pct:150 },
f13: { desc:'全队主动攻击，每只200%伤害', pct:200 },
f14: { desc:'暴击伤害+60%+必暴击2回合', pct:60, critDur:2 },
f15: { desc:'眩晕3回合+火伤×4倍', stunDur:3, pct:300 },
f16: { desc:'700%火属性爆裂伤害+灼烧40点3回合', pct:700, dotDmg:40 },
f17: { desc:'全队攻击+65%持续4回合', pct:65, dur:4 },
f18: { desc:'每段Combo暴击率+15%', pct:15 },
f19: { desc:'火伤无视全防+伤害×2倍', dmgMul:100 },
f20: { desc:'残血时伤害+120%', pct:120 },
// --- 土属性 ---
```

```
  e1: { desc:'全部水珠转心珠+心珠回复+40%', heartBoost:40 },
  e2: { desc:'整行变土珠+额外随机3颗变土珠', extra:3 },
  e3: { desc:'护盾55点+减伤15%持续1回合', val:55, reducePct:15 },
  e4: { desc:'眩晕2回合+期间受伤+30%', dur:2, extraDmgPct:30 },
  e5: { desc:'下次土属性伤害×3.5倍', pct:250 },
  e6: { desc:'血量上限+40%+立即回满', pct:40 },
  e7: { desc:'免疫所有控制3回合+护盾20点', dur:3, shieldVal:20 },
  e8: { desc:'反弹40%伤害3回合', pct:40, dur:3 },
  e9: { desc:'500%土属性直接伤害', pct:500 },
  e10: { desc:'全队防御+75%持续4回合', pct:75, dur:4 },
  e11: { desc:'随机12颗变土珠+土珠概率大增', count:12 },
  e12: { desc:'连续攻击4次，每次140%土伤害', hits:4, pct:140 },
  e13: { desc:'全队防御+55%持续3回合', pct:55, dur:3 },
  e14: { desc:'眩晕2回合+破甲（防御归零）+受伤+20%', stunDur:2, extraDmgPct:20 },
  e15: { desc:'全部水珠转土珠+土珠概率大增+土伤+25%', dmgBoost:25 },
  e16: { desc:'眩晕2回合+破甲（防御归零）', stunDur:2 },
  e17: { desc:'十字形全变土珠+土属性伤害+40%', dmgBoost:40 },
  e18: { desc:'700%土属性爆裂伤害+眩晕1回合', pct:700, stunDur:1 },
  e19: { desc:'全部水珠转土珠+全队防御+30%', defBoost:30 },
  e20: { desc:'眩晕3回合+土伤×4倍', stunDur:3, pct:300 },
}
// 判断宠物当前星级是否已解锁技能（★1无技能，★2+解锁）
function petHasSkill(pet) {
  return (pet.star || 1) >= 2 && !!pet.skill
}
// 获取宠物当前技能描述（★1无技能，★2基础技能，★3强化技能）
function getPetSkillDesc(pet) {
  const star = pet.star || 1
  if (star < 2) return ''  // ★1无技能
  if (star >= 3 && STAR3_SKILL_OVERRIDE[pet.id]) {
    return STAR3_SKILL_OVERRIDE[pet.id].desc
  }
  return pet.skill ? pet.skill.desc : ''
}
// 获取★3强化数据（若有）
function getStar3Override(petId) {
  return STAR3_SKILL_OVERRIDE[petId] || null
}
// ===== 星级融合系统常量 =====
const MAX_STAR = 3         // 最高星级
const STAR_ATK_MUL = 1.3    // 每升1星ATK倍率
const STAR_SKILL_MUL = 1.25 // 每升1星技能数值倍率
// 获取宠物星级加成后的ATK
function getPetStarAtk(pet) {
  const star = pet.star || 1
  return Math.floor(pet.atk * Math.pow(STAR_ATK_MUL, star - 1))
}
// 获取宠物星级技能数值倍率（★1无技能，★2=1.0基础，★3=1.25强化）
function getPetStarSkillMul(pet) {
  const star = pet.star || 1
```

```
    c.textAlign = 'center'; c.textBaseline = 'top'
    c.font = `bold ${10*S}px "PingFang SC",sans-serif`
    const displayName = pet.name.length > 4 ? pet.name.slice(0, 4) + '…' : pet.name
    c.strokeStyle = 'rgba(0,0,0,0.7)'; c.lineWidth = 3 * S
    c.strokeText(displayName, cx + cw / 2, nameY)
    c.fillStyle = isSelected ? '#ffd700' : '#fff'
    c.fillText(displayName, cx + cw / 2, nameY)
    // 星级
    const starY = nameY + 14 * S
    c.font = `${10*S}px "PingFang SC",sans-serif`
    let starStr = ''
    for (let si = 0; si < 3; si++) starStr += '★'
    const totalStarW = c.measureText(starStr).width
    let starX = cx + cw / 2 - totalStarW / 2
    c.textAlign = 'left'
    for (let si = 0; si < 3; si++) {
      c.fillStyle = si < pp.star ? '#ffd700' : 'rgba(120,120,120,0.5)'
      c.fillText('★', starX, starY)
      starX += c.measureText('★').width
    }
    c.textAlign = 'center'
    // ATK（红色）
    const atkY = starY + 14 * S
    c.font = `bold ${9*S}px "PingFang SC",sans-serif`
    c.strokeStyle = 'rgba(0,0,0,0.7)'; c.lineWidth = 2.5 * S
    c.strokeText(`ATK:${atk}`, cx + cw / 2, atkY)
    c.fillStyle = '#FF6B6B'
    c.fillText(`ATK:${atk}`, cx + cw / 2, atkY)
  }
  // 滚动限制
  const totalRows = Math.ceil(sorted.length / cols)
  const totalH = totalRows * (ch + gap) + gap
  const viewH = listBottom - listTop
  const maxScroll = Math.max(0, totalH - viewH)
  if (_scrollY < -maxScroll) _scrollY = -maxScroll
  if (_scrollY > 0) _scrollY = 0
  c.restore()
}
// ===== 触摸 =====
function tStageTeam(g, x, y, type) {
  if (type === 'start') {
    _touchStartY = y; _touchLastY = y; _touchStartX = x; _scrolling = false
    _holdStartTime = Date.now()
    _holdTarget = null
    // 记录按下的宠物卡片
    for (const item of _rects.petCardRects) {
      if (g._hitRect(x, y, ...item.rect)) {
        _holdTarget = { petId: item.petId }
        break
      }
```

```
27576   }
27577   return
27578 }
27579 if (type === 'move') {
27580   const dy = y - _touchLastY; _touchLastY = y
27581   if (Math.abs(y - _touchStartY) > 5 * V.S || Math.abs(x - _touchStartX) > 5 * V.S) {
27582     _scrolling = true
27583     _holdTarget = null // 移动则取消长按
27584   }
27585   if (_scrolling) _scrollY += dy
27586   return
27587 }
27588 if (type !== 'end') return
27589 // 长按检测（≥500ms 且未滚动 → 跳转宠物详情全屏页）
27590 const elapsed = Date.now() - _holdStartTime
27591 if (elapsed >= 500 && !_scrolling && _holdTarget) {
27592   g._petDetailId = _holdTarget.petId
27593   g._petDetailReturnScene = 'stageTeam'
27594   _holdTarget = null
27595   g.setScene('petDetail')
27596   return
27597 }
27598 _holdTarget = null
27599 if (_scrolling) return
27600 const stage = getStageById(g._selectedStageId)
27601 if (!stage) return
27602 const selected = g._stageTeamSelected || (g._stageTeamSelected = [])
27603 // 开始战斗
27604 if (_rects.startBtnRect && g._hitRect(x, y, ..._rects.startBtnRect)) {
27605   // 保存编队
27606   g.storage.saveStageteam(selected)
27607   // 检查体力
27608   if (g.storage.currentStamina < stage.staminaCost) {
27609     g._toastMsg = '体力不足'; return
27610   }
27611   if (!g.storage.canChallengeStage(g._selectedStageId, stage.dailyLimit)) {
27612     g._toastMsg = '今日挑战次数已用完'; return
27613   }
27614   const stageMgr = require('../engine/stageManager')
27615   stageMgr.startStage(g, g._selectedStageId, selected)
27616   return
27617 }
27618 // 返回（回到关卡信息页，同时保存编队）
27619 if (_rects.backBtnRect && g._hitRect(x, y, ..._rects.backBtnRect)) {
27620   g.storage.saveStageteam(selected)
27621   g.setScene('stageInfo')
27622   return
27623 }
27624 // 槽位点击（取消选择该宠物）
27625 for (let i = 0; i < _rects.slotRects.length; i++) {
```

```
27626     if (i < selected.length && g._hitRect(x, y, ..._rects.slotRects[i])) {
27627       selected.splice(i, 1)
27628       return
27629     }
27630   }
27631   // 筛选标签
27632   for (const f of _rects.filterRects) {
27633     if (g._hitRect(x, y, ...f.rect)) {
27634       g._stageTeamFilter = f.key; _scrollY = 0
27635       return
27636     }
27637   }
27638   // 宠物卡片（选入/取消）
27639   for (const item of _rects.petCardRects) {
27640     if (g._hitRect(x, y, ...item.rect)) {
27641       const idx = selected.indexOf(item.petId)
27642       if (idx >= 0) {
27643         selected.splice(idx, 1)
27644       } else if (selected.length < stage.teamSize.max) {
27645         selected.push(item.petId)
27646       }
27647       return
27648     }
27649   }
27650 }
27651 // ===== 工具 =====
27652 function resetScroll() { _scrollY = 0 }
27653 module.exports = { rStageTeam, tStageTeam, resetScroll }
27654 /**
27655  * 首页渲染 — 场景式大厅 v3
27656  * 布局：顶栏 / 场景区（背景+插画+Logo） / 灵宠展示 / 开始按钮区 / 模式切换浮钮 / 7标签底部导航
27657  */
27658 const V = require('./env')
27659 const P = require('../platform')
27660 const { BAR_ITEMS, getLayout, drawBottomBar } = require('./bottomBar')
27661 // 模式配置
27662 const MODE_CFG = {
27663   tower: { name: '通天塔',  img: 'assets/ui/tower_rogue.png', icon: '',  switchKey: 'stage' },
27664   stage: { name: '灵兽秘境', img: 'assets/ui/gate_stage.png', icon: '?', switchKey: 'tower' },
27665 }
27666 // ===== ZONE 1: 顶栏（已隐藏文字，保留宝箱区域占位）=====
27667 function drawTopBar(g) {
27668   // 顶栏文字（最佳层数/挑战次数）已移除，顶栏不再绘制
27669 }
27670 // ===== ZONE 2: 场景区（背景 + 插画 + Logo）=====
27671 function drawSceneArea(g) {
27672   const { ctx, R, W, S } = V
27673   const L = getLayout()
27674   R.drawHomeBg(g.af)
27675   const mode = g.titleMode || 'tower'
```

```
  const imgPath = MODE_CFG[mode].img
  const towerImg = R.getImg(imgPath)
  const sceneH = L.petRowY - L.topBarBottom
  if (towerImg && towerImg.width > 0) {
    // 以场景高度为基准，使塔图填满纵向空间（最宽不超过 92%W）
    const targetH = sceneH * 0.88
    const ratioW = towerImg.width / towerImg.height
    const imgW = Math.min(targetH * ratioW, W * 0.92)
    const imgH = imgW / ratioW
    const imgX = (W - imgW) / 2
    // 底部对齐到 petRowY + 14S（与按钮略微叠压）
    const imgY = L.petRowY - imgH + 14 * S
    ctx.drawImage(towerImg, imgX, imgY, imgW, imgH)
  } else {
    // 无素材 fallback
    const emoji = mode === 'tower' ? '?' : '?'
    ctx.save()
    ctx.font = `${80*S}px "PingFang SC",sans-serif`
    ctx.textAlign = 'center'; ctx.textBaseline = 'middle'
    ctx.globalAlpha = mode === 'stage' ? 0.35 : 0.6
    ctx.fillText(emoji, W / 2, L.topBarBottom + sceneH * 0.45)
    ctx.globalAlpha = 1
    ctx.restore()
  }
}
// ===== ZONE 3: 人物头像+等级（左上角常驻）=====
function drawAvatarWidget(g) {
  const { ctx, R, W, S, safeTop } = V
  const cult = g.storage.cultivation
  const level = (cult && cult.level) || 0
  const selectedId = g.storage.selectedAvatar || 'boy1'
  const CHAR_MAP = {
    boy1:  'assets/hero/char_boy1.png',
    girl1: 'assets/hero/char_girl1.png',
    boy2:  'assets/hero/char_boy2.png',
    girl2: 'assets/hero/char_girl2.png',
    boy3:  'assets/hero/char_boy3.png',
    girl3: 'assets/hero/char_girl3.png',
  }
  const sitPath = CHAR_MAP[selectedId] || CHAR_MAP.boy1
  const iconSz = 44 * S
  const iconX = 10 * S
  const iconY = safeTop + 6 * S
  const iconCX = iconX + iconSz / 2
  const iconCY = iconY + iconSz / 2
  ctx.save()
  // 1. 头像圆形裁剪
  const img = R.getImg(sitPath)
  if (img && img.width > 0) {
    ctx.save()
```

```
27726   ctx.beginPath(); ctx.arc(iconCX, iconCY, iconSz / 2, 0, Math.PI * 2); ctx.clip()
27727   const cropSz = Math.min(img.width, img.height * 0.60)
27728   const srcX = (img.width - cropSz) / 2
27729   ctx.drawImage(img, srcX, 0, cropSz, cropSz, iconX, iconY, iconSz, iconSz)
27730   ctx.restore()
27731 } else {
27732   ctx.fillStyle = 'rgba(180,150,80,0.5)'
27733   ctx.beginPath(); ctx.arc(iconCX, iconCY, iconSz / 2, 0, Math.PI * 2); ctx.fill()
27734 }
27735 // 2. 圆形头像框
27736 const frameImg = R.getImg('assets/ui/frame_avatar.png')
27737 if (frameImg && frameImg.width > 0) {
27738   const frameSz = iconSz * 1.22
27739   ctx.drawImage(frameImg, iconCX - frameSz / 2, iconCY - frameSz / 2, frameSz, frameSz)
27740 } else {
27741   ctx.beginPath(); ctx.arc(iconCX, iconCY, iconSz / 2, 0, Math.PI * 2)
27742   ctx.strokeStyle = 'rgba(220,180,60,0.9)'; ctx.lineWidth = 2 * S; ctx.stroke()
27743 }
27744 // 3. 等级标签（右下角偏右，不遮头像）
27745 const lvText = `Lv.${level}`
27746 ctx.font = `bold ${12*S}px "PingFang SC",sans-serif`
27747 ctx.textAlign = 'center'; ctx.textBaseline = 'middle'
27748 const lvTxtW = ctx.measureText(lvText).width
27749 const padX = 7 * S
27750 const capW = lvTxtW + padX * 2
27751 const capH = 20 * S
27752 const capR = capH / 2
27753 // 胶囊左端从头像右边缘开始，整体在头像外侧偏下
27754 const capCX = iconX + iconSz + capW / 2 - 4 * S
27755 const capCY = iconCY + iconSz * 0.28
27756 const capX = capCX - capW / 2
27757 const capY = capCY - capH / 2
27758 ctx.beginPath()
27759 ctx.moveTo(capX + capR, capY); ctx.lineTo(capX + capW - capR, capY)
27760 ctx.quadraticCurveTo(capX + capW, capY, capX + capW, capY + capR)
27761 ctx.lineTo(capX + capW, capY + capH - capR)
27762 ctx.quadraticCurveTo(capX + capW, capY + capH, capX + capW - capR, capY + capH)
27763 ctx.lineTo(capX + capR, capY + capH)
27764 ctx.quadraticCurveTo(capX, capY + capH, capX, capY + capH - capR)
27765 ctx.lineTo(capX, capY + capR)
27766 ctx.quadraticCurveTo(capX, capY, capX + capR, capY)
27767 ctx.closePath()
27768 ctx.fillStyle = 'rgba(0,0,0,0.52)'; ctx.fill()
27769 ctx.strokeStyle = 'rgba(220,185,60,0.6)'; ctx.lineWidth = 1 * S; ctx.stroke()
27770 ctx.shadowColor = 'rgba(0,0,0,0.5)'; ctx.shadowBlur = 2 * S; ctx.shadowOffsetY = 1 * S
27771 ctx.strokeStyle = 'rgba(30,15,0,0.7)'; ctx.lineWidth = 2.5 * S
27772 ctx.strokeText(lvText, capCX, capCY)
27773 ctx.shadowColor = 'transparent'
27774 ctx.fillStyle = '#fff8cc'
27775 ctx.fillText(lvText, capCX, capCY)
```

```
  ctx.restore()
}
// ===== ZONE 3b: 体力显示（左上角，仅灵兽秘境模式）=====
function drawStaminaBar(g) {
  if ((g.titleMode || 'tower') !== 'stage') return
  const { ctx, R, S, safeTop } = V
  const stamina = g.storage.currentStamina
  const maxSt = g.storage.maxStamina
  const iconSz = 32 * S
  const iconX = 10 * S
  // 头像高度 44S + 间距6S 之后
  const centerY = safeTop + 6 * S + 44 * S + 10 * S + iconSz / 2
  const stIcon = R.getImg('assets/ui/icon_stamina.png')
  // 量文字
  const txt = `${stamina}/${maxSt}`
  ctx.font = `bold ${13*S}px "PingFang SC",sans-serif`
  ctx.textAlign = 'left'; ctx.textBaseline = 'middle'
  const txtW = ctx.measureText(txt).width
  const padX = 8 * S
  const capH = 26 * S, capR = capH / 2
  const txtX = iconX + iconSz + 4 * S
  const capX = iconX + iconSz * 0.38
  const capW = txtX + txtW + padX - capX
  const capY = centerY - capH / 2
  ctx.save()
  // 画胶囊
  ctx.beginPath()
  ctx.moveTo(capX + capR, capY); ctx.lineTo(capX + capW - capR, capY)
  ctx.quadraticCurveTo(capX + capW, capY, capX + capW, capY + capR)
  ctx.lineTo(capX + capW, capY + capH - capR)
  ctx.quadraticCurveTo(capX + capW, capY + capH, capX + capW - capR, capY + capH)
  ctx.lineTo(capX + capR, capY + capH)
  ctx.quadraticCurveTo(capX, capY + capH, capX, capY + capH - capR)
  ctx.lineTo(capX, capY + capR)
  ctx.quadraticCurveTo(capX, capY, capX + capR, capY)
  ctx.closePath()
  ctx.fillStyle = 'rgba(0,0,0,0.45)'; ctx.fill()
  // 数字
  ctx.fillStyle = '#fff'
  ctx.fillText(txt, txtX, centerY)
  // 图标压上去
  if (stIcon && stIcon.width > 0) {
    ctx.drawImage(stIcon, iconX, centerY - iconSz / 2, iconSz, iconSz)
  } else {
    ctx.fillStyle = '#3aaeff'; ctx.font = `${14*S}px sans-serif`
    ctx.textBaseline = 'middle'; ctx.textAlign = 'left'
    ctx.fillText('', iconX, centerY)
  }
  ctx.restore()
}
```

```
// ===== ZONE 4: 开始按钮区 =====
function drawStartBtn(g) {
  const { ctx, R, W, S } = V
  const L = getLayout()
  const mode = g.titleMode || 'tower'
  ctx.save()
  if (mode === 'tower') {
    const btnW = W * 0.60
    const btnH = L.startBtnH
    const btnX = (W - btnW) / 2
    const btnY = L.startBtnY
    // 使用 btn_start.png 资源，fallback 到渐变色
    const btnImg = R.getImg('assets/ui/btn_start.png')
    if (btnImg && btnImg.width > 0) {
      ctx.drawImage(btnImg, btnX, btnY, btnW, btnH)
    } else {
      const grad = ctx.createLinearGradient(btnX, btnY, btnX, btnY + btnH)
      grad.addColorStop(0, '#f5d98a')
      grad.addColorStop(0.5, '#d4a84b')
      grad.addColorStop(1, '#b8862d')
      ctx.fillStyle = grad
      R.rr(btnX, btnY, btnW, btnH, btnH * 0.4); ctx.fill()
    }
    // 按钮文字叠在图片上
    const hasSave = g.storage.hasSavedRun()
    ctx.fillStyle = '#5a2d0c'
    ctx.font = `bold ${15*S}px "PingFang SC",sans-serif`
    ctx.textAlign = 'center'; ctx.textBaseline = 'middle'
    ctx.fillText(hasSave ? '继续挑战' : '开始挑战', btnX + btnW / 2, btnY + btnH / 2)
    g._startBtnRect = [btnX, btnY, btnW, btnH]
    // 进度文字
    let progressText = ''
    if (hasSave) {
      const saved = g.storage.loadRunState()
      progressText = `继续第 ${saved.floor} 层 · 历史最高 ${g.storage.bestFloor} 层`
    } else {
      const best = g.storage.bestFloor
      progressText = best > 0 ? `历史最高第 ${best} 层` : '开始你的第一次冒险'
    }
    ctx.fillStyle = 'rgba(80,50,20,0.7)'
    ctx.font = `${10*S}px "PingFang SC",sans-serif`
    ctx.textBaseline = 'middle'
    ctx.fillText(progressText, W / 2, L.progressY + L.progressH / 2)
  } else {
    // 灵兽秘境模式
    const btnW = W * 0.55
    const btnH = L.startBtnH
    const btnX = (W - btnW) / 2
    const btnY = L.startBtnY
    const poolCount = g.storage.petPoolCount
```

```
    const canPlay = poolCount >= 5
    if (canPlay) {
      // 可游玩：使用 btn_start.png，fallback 蓝色渐变
      const btnImg = R.getImg('assets/ui/btn_start.png')
      if (btnImg && btnImg.width > 0) {
        ctx.drawImage(btnImg, btnX, btnY, btnW, btnH)
      } else {
        const grad = ctx.createLinearGradient(btnX, btnY, btnX, btnY + btnH)
        grad.addColorStop(0, '#8ac8ff'); grad.addColorStop(1, '#4a8acc')
        ctx.fillStyle = grad
        R.rr(btnX, btnY, btnW, btnH, btnH * 0.4); ctx.fill()
      }
      ctx.fillStyle = '#1a2a3c'
      ctx.font = `bold ${15*S}px "PingFang SC",sans-serif`
      ctx.textAlign = 'center'; ctx.textBaseline = 'middle'
      ctx.fillText('选择关卡', btnX + btnW / 2, btnY + btnH / 2)
      g._startBtnRect = [btnX, btnY, btnW, btnH]
    } else {
      // 未解锁
      ctx.fillStyle = 'rgba(100,100,120,0.35)'
      R.rr(btnX, btnY, btnW, btnH, btnH * 0.4); ctx.fill()
      ctx.fillStyle = 'rgba(140,140,160,0.6)'
      ctx.font = `bold ${14*S}px "PingFang SC",sans-serif`
      ctx.textAlign = 'center'; ctx.textBaseline = 'middle'
      ctx.fillText(`? 灵宠池需 ${5 - poolCount} 只解锁`, btnX + btnW / 2, btnY + btnH / 2)
      g._startBtnRect = null
    }
  }
  ctx.restore()
}
// ===== ZONE 4b: 开始/继续确认弹窗 =====
function drawTitleStartDialog(g) {
  if (!g.showTitleStartDialog) return
  const { ctx, R, W, H, S } = V
  const hasSave = g.storage.hasSavedRun()
  // 遮罩
  ctx.save()
  ctx.fillStyle = 'rgba(0,0,0,0.55)'
  ctx.fillRect(0, 0, W, H)
  // 面板
  const pw = W * 0.82
  const ph = hasSave ? 190 * S : 150 * S
  const px = (W - pw) / 2
  const py = (H - ph) / 2
  const panelImg = R.getImg('assets/ui/info_panel_bg.png')
  if (panelImg && panelImg.width > 0) {
    ctx.drawImage(panelImg, px, py, pw, ph)
  } else {
    const rad = 14 * S
    ctx.fillStyle = 'rgba(248,242,230,0.97)'
```

```
27926    R.rr(px, py, pw, ph, rad); ctx.fill()
27927    ctx.strokeStyle = 'rgba(201,168,76,0.4)'; ctx.lineWidth = 1.5 * S
27928    R.rr(px, py, pw, ph, rad); ctx.stroke()
27929  }
27930  ctx.textAlign = 'center'
27931  if (hasSave) {
27932    const saved = g.storage.loadRunState()
27933    // 标题
27934    ctx.fillStyle = '#6B5014'
27935    ctx.font = `bold ${16*S}px "PingFang SC",sans-serif`
27936    ctx.fillText('通天塔', px + pw * 0.5, py + 30 * S)
27937    // 进度信息
27938    ctx.fillStyle = '#8a6a20'
27939    ctx.font = `${12*S}px "PingFang SC",sans-serif`
27940    ctx.fillText(`进度：第 ${saved.floor} 层`, px + pw * 0.5, py + 58 * S)
27941    ctx.fillStyle = '#7B7060'
27942    ctx.font = `${10*S}px "PingFang SC",sans-serif`
27943    ctx.fillText(`历史最高第 ${g.storage.bestFloor} 层`, px + pw * 0.5, py + 78 * S)
27944    // 红字提醒（将原二次确认提示并入本弹窗）
27945    ctx.fillStyle = '#C0392B'
27946    ctx.font = `bold ${11*S}px "PingFang SC",sans-serif`
27947    ctx.fillText('重新挑战将清空当前进度记录', px + pw * 0.5, py + 98 * S)
27948    // 按钮：左重新挑战，右继续挑战
27949    const btnW = pw * 0.36, btnH = 34 * S, gap = 12 * S
27950    const btn1X = px + pw * 0.5 - btnW - gap * 0.5
27951    const btn2X = px + pw * 0.5 + gap * 0.5
27952    const btnY = py + 122 * S
27953    R.drawDialogBtn(btn1X, btnY, btnW, btnH, '重新挑战', 'cancel')
27954    g._dialogStartRect = [btn1X, btnY, btnW, btnH]
27955    R.drawDialogBtn(btn2X, btnY, btnW, btnH, '继续挑战', 'confirm')
27956    g._dialogContinueRect = [btn2X, btnY, btnW, btnH]
27957    g._dialogCancelRect = null
27958  } else {
27959    // 无存档
27960    ctx.fillStyle = '#6B5014'
27961    ctx.font = `bold ${16*S}px "PingFang SC",sans-serif`
27962    ctx.fillText('通天塔', px + pw * 0.5, py + 30 * S)
27963    ctx.fillStyle = '#7B7060'
27964    ctx.font = `${11*S}px "PingFang SC",sans-serif`
27965    ctx.fillText('踏入通天塔，开始你的修炼之旅', px + pw * 0.5, py + 64 * S)
27966    const btnW = pw * 0.36, btnH = 34 * S, gap = 12 * S
27967    const btn1X = px + pw * 0.5 - btnW - gap * 0.5
27968    const btn2X = px + pw * 0.5 + gap * 0.5
27969    const btnY = py + 90 * S
27970    R.drawDialogBtn(btn1X, btnY, btnW, btnH, '取消', 'cancel')
27971    g._dialogCancelRect = [btn1X, btnY, btnW, btnH]
27972    R.drawDialogBtn(btn2X, btnY, btnW, btnH, '开始挑战', 'confirm')
27973    g._dialogStartRect = [btn2X, btnY, btnW, btnH]
27974    g._dialogContinueRect = null
27975  }
```

```
27976  ctx.restore()
27977 }
27978 // ===== 左侧居中模式切换浮钮 =====
27979 function drawModeSwitchBtn(g) {
27980  const { ctx, R, W, H, S, safeTop } = V
27981  const L = getLayout()
27982  const mode = g.titleMode || 'tower'
27983  const targetMode = MODE_CFG[mode].switchKey
27984  const targetCfg = MODE_CFG[targetMode]
27985  // 竖排布局：图标 + 文字，贴屏幕左边缘垂直居中（场景区中央）
27986  const iconSize  = 44 * S
27987  const labelSize = 11 * S
27988  const vGap      = 4 * S
27989  const btnW      = iconSize + 8 * S
27990  const btnH      = iconSize + vGap + labelSize + 8 * S
27991  // 垂直居中于场景区（顶栏底到开始按钮之间）
27992  const sceneTop    = safeTop + 36 * S
27993  const sceneBottom = L.startBtnY
27994  const btnCY       = (sceneTop + sceneBottom) / 2
27995  const btnX        = 0
27996  const btnY        = btnCY - btnH / 2
27997  const icx         = btnX + btnW / 2
27998  const icy         = btnY + 8 * S + iconSize / 2
27999  ctx.save()
28000  // 半透明胶囊背景（右侧半圆，左侧贴边）
28001  const bgR = btnW / 2
28002  ctx.beginPath()
28003  ctx.moveTo(btnX, btnY)
28004  ctx.lineTo(btnX + btnW - bgR, btnY)
28005  ctx.quadraticCurveTo(btnX + btnW, btnY, btnX + btnW, btnY + bgR)
28006  ctx.lineTo(btnX + btnW, btnY + btnH - bgR)
28007  ctx.quadraticCurveTo(btnX + btnW, btnY + btnH, btnX + btnW - bgR, btnY + btnH)
28008  ctx.lineTo(btnX, btnY + btnH)
28009  ctx.closePath()
28010  ctx.fillStyle = 'rgba(255,245,220,0.82)'; ctx.fill()
28011  ctx.strokeStyle = 'rgba(200,165,60,0.7)'; ctx.lineWidth = 1.5 * S; ctx.stroke()
28012  // 切换图标
28013  const btnImg = R.getImg('assets/ui/btn_mode_switch.png')
28014  if (btnImg && btnImg.width > 0) {
28015   const drawSz = iconSize * 0.75
28016   ctx.save()
28017   ctx.shadowColor = 'rgba(0,0,0,0.3)'
28018   ctx.shadowBlur = 3 * S; ctx.shadowOffsetY = 1.5 * S
28019   ctx.drawImage(btnImg, icx - drawSz / 2, icy - drawSz / 2, drawSz, drawSz)
28020   ctx.restore()
28021  } else {
28022   ctx.fillStyle = 'rgba(160,110,30,0.9)'
28023   ctx.font = `${iconSize * 0.5}px sans-serif`
28024   ctx.textAlign = 'center'; ctx.textBaseline = 'middle'
28025   ctx.fillText('', icx, icy)
```

```
28026  }
28027  // 文字标签
28028  const labelY = icy + iconSize / 2 + vGap + labelSize * 0.5
28029  ctx.fillStyle = '#7a5520'
28030  ctx.strokeStyle = 'rgba(255,245,220,0.8)'
28031  ctx.lineWidth = 2 * S
28032  ctx.font = `bold ${labelSize}px "PingFang SC",sans-serif`
28033  ctx.textAlign = 'center'; ctx.textBaseline = 'middle'
28034  ctx.strokeText(targetCfg.name, icx, labelY)
28035  ctx.fillText(targetCfg.name, icx, labelY)
28036  g._modeSwitchRect = [btnX, btnY, btnW, btnH]
28037  ctx.restore()
28038 }
28039 // ===== 右下角侧边栏复访入口（抖音专属） =====
28040 function _isFromSidebar() {
28041  const info = GameGlobal.__launchInfo || {}
28042  return info.scene === '021036' && info.launch_from === 'homepage' && info.location === 'sidebar...
28043 }
28044 function drawSidebarBtn(g) {
28045  if (!P.isDouyin || !GameGlobal.__sidebarSupported) { g._sidebarBtnRect = null; return }
28046  const { ctx, R, W, S } = V
28047  const L = getLayout()
28048  const iconSize = L.modeSwitchH * 0.72
28049  const labelSize = 11 * S
28050  const vGap = 3 * S
28051  const btnH = iconSize + vGap + labelSize
28052  const btnW = iconSize
28053  const btnX = W - L.pad - btnW + 4 * S
28054  const btnY = L.modeSwitchY - 6 * S
28055  ctx.save()
28056  ctx.fillStyle = 'rgba(20,18,38,0.75)'
28057  R.rr(btnX, btnY, btnW, iconSize, 8 * S); ctx.fill()
28058  ctx.strokeStyle = 'rgba(200,180,120,0.45)'; ctx.lineWidth = 1.5 * S
28059  R.rr(btnX, btnY, btnW, iconSize, 8 * S); ctx.stroke()
28060  const claimed = g.storage.sidebarRewardClaimedToday
28061  const fromSB = _isFromSidebar()
28062  const hasReward = fromSB && !claimed
28063  const icx = btnX + btnW / 2
28064  const icy = btnY + iconSize / 2
28065  ctx.font = `${iconSize * 0.44}px sans-serif`
28066  ctx.textAlign = 'center'; ctx.textBaseline = 'middle'
28067  ctx.fillStyle = hasReward ? 'rgba(255,80,80,0.95)' : 'rgba(255,210,80,0.92)'
28068  ctx.fillText(hasReward ? '?' : '?', icx, icy)
28069  if (hasReward) {
28070   const dotR = 5 * S
28071   ctx.beginPath()
28072   ctx.arc(btnX + btnW - 2 * S, btnY + 2 * S, dotR, 0, Math.PI * 2)
28073   ctx.fillStyle = '#ff3333'
28074   ctx.fill()
28075  }
```

```
28076   const labelY = btnY + iconSize + vGap + labelSize * 0.5
28077   ctx.fillStyle = 'rgba(255,235,160,0.95)'
28078   ctx.strokeStyle = 'rgba(20,10,40,0.7)'
28079   ctx.lineWidth = 2.5 * S
28080   ctx.font = `bold ${labelSize}px "PingFang SC",sans-serif`
28081   ctx.textAlign = 'center'; ctx.textBaseline = 'middle'
28082   ctx.strokeText('侧边栏', icx, labelY)
28083   ctx.fillText('侧边栏', icx, labelY)
28084   g._sidebarBtnRect = [btnX, btnY, btnW, btnH]
28085   ctx.restore()
28086 }
28087 // ===== 侧边栏复访引导弹窗 =====
28088 function drawSidebarPanel(g) {
28089   if (!g.showSidebarPanel) return
28090   const { ctx, R, W, H, S } = V
28091   const fromSB = _isFromSidebar()
28092   const claimed = g.storage.sidebarRewardClaimedToday
28093   const canClaim = fromSB && !claimed
28094   ctx.save()
28095   ctx.fillStyle = 'rgba(0,0,0,0.6)'
28096   ctx.fillRect(0, 0, W, H)
28097   const pw = W * 0.78, ph = 200 * S
28098   const px = (W - pw) / 2, py = (H - ph) / 2 - 20 * S
28099   const bgGrad = ctx.createLinearGradient(px, py, px, py + ph)
28100   bgGrad.addColorStop(0, 'rgba(45,30,80,0.97)')
28101   bgGrad.addColorStop(1, 'rgba(25,15,50,0.97)')
28102   ctx.fillStyle = bgGrad
28103   R.rr(px, py, pw, ph, 14 * S); ctx.fill()
28104   ctx.strokeStyle = 'rgba(220,185,110,0.6)'; ctx.lineWidth = 2 * S
28105   R.rr(px, py, pw, ph, 14 * S); ctx.stroke()
28106   ctx.fillStyle = '#ffe8a0'
28107   ctx.font = `bold ${16 * S}px "PingFang SC",sans-serif`
28108   ctx.textAlign = 'center'; ctx.textBaseline = 'middle'
28109   ctx.fillText('侧边栏复访奖励', W / 2, py + 28 * S)
28110   ctx.fillStyle = 'rgba(255,255,255,0.8)'
28111   ctx.font = `${12 * S}px "PingFang SC",sans-serif`
28112   if (canClaim) {
28113     ctx.fillText('你从侧边栏进入了游戏！', W / 2, py + 56 * S)
28114     ctx.fillStyle = '#ffcc44'
28115     ctx.font = `bold ${14 * S}px "PingFang SC",sans-serif`
28116     ctx.fillText('? 奖励：体力 +30', W / 2, py + 84 * S)
28117     const btnW2 = pw * 0.55, btnH2 = 38 * S
28118     const btnX2 = (W - btnW2) / 2, btnY2 = py + ph - 56 * S
28119     R.drawDialogBtn(btnX2, btnY2, btnW2, btnH2, '领取奖励', 'confirm')
28120     g._sidebarClaimRect = [btnX2, btnY2, btnW2, btnH2]
28121     g._sidebarGoRect = null
28122   } else if (claimed) {
28123     ctx.fillText('今日奖励已领取，明天再来吧~', W / 2, py + 64 * S)
28124     ctx.fillStyle = '#aaa'
28125     ctx.font = `${11 * S}px "PingFang SC",sans-serif`
```

```
    ctx.fillText('每天从抖音首页侧边栏进入游戏', W / 2, py + 90 * S)
    ctx.fillText('即可领取体力奖励', W / 2, py + 108 * S)
    g._sidebarClaimRect = null
    g._sidebarGoRect = null
  } else {
    ctx.fillText('① 点击下方按钮前往侧边栏', W / 2, py + 54 * S)
    ctx.fillText('② 在侧边栏找到本游戏并点击进入', W / 2, py + 74 * S)
    ctx.fillText('③ 返回后即可领取体力奖励', W / 2, py + 94 * S)
    const btnW2 = pw * 0.55, btnH2 = 38 * S
    const btnX2 = (W - btnW2) / 2, btnY2 = py + ph - 56 * S
    R.drawDialogBtn(btnX2, btnY2, btnW2, btnH2, '去首页侧边栏', 'confirm')
    g._sidebarGoRect = [btnX2, btnY2, btnW2, btnH2]
    g._sidebarClaimRect = null
  }
  const closeSize = 28 * S
  const closeX = px + pw - closeSize - 4 * S, closeY = py + 4 * S
  ctx.fillStyle = 'rgba(255,255,255,0.5)'
  ctx.font = `${18 * S}px sans-serif`
  ctx.textAlign = 'center'; ctx.textBaseline = 'middle'
  ctx.fillText('', closeX + closeSize / 2, closeY + closeSize / 2)
  g._sidebarCloseRect = [closeX, closeY, closeSize, closeSize]
  ctx.restore()
}
// ===== 「更多」弹出面板 =====
function drawMorePanel(g) {
  if (!g.showMorePanel) return
  const { ctx, R, W, H, S } = V
  const L = getLayout()
  ctx.save()
  ctx.fillStyle = 'rgba(0,0,0,0.5)'
  ctx.fillRect(0, 0, W, H)
  const panelH = 200 * S + L.safeBottom
  const panelY = H - panelH
  const rad = 16 * S
  ctx.fillStyle = 'rgba(20,18,38,0.95)'
  ctx.beginPath()
  ctx.moveTo(0, panelY + rad)
  ctx.arcTo(0, panelY, rad, panelY, rad)
  ctx.lineTo(W - rad, panelY)
  ctx.arcTo(W, panelY, W, panelY + rad, rad)
  ctx.lineTo(W, H); ctx.lineTo(0, H)
  ctx.closePath(); ctx.fill()
  ctx.fillStyle = '#ffd700'
  ctx.font = `bold ${14*S}px "PingFang SC",sans-serif`
  ctx.textAlign = 'center'; ctx.textBaseline = 'top'
  ctx.fillText('更多', W / 2, panelY + 16 * S)
  const items = [
    { key: 'sfx',      label: '音效',     toggle: g.storage.settings.sfxOn },
    { key: 'bgm',      label: '背景音乐', toggle: g.storage.settings.bgmOn },
    { key: 'feedback', label: '意见反馈', toggle: null },
```

```
28176  ]
28177  const itemH = 44 * S
28178  const listX = 24 * S
28179  const listW = W - 48 * S
28180  let itemY = panelY + 44 * S
28181  g._morePanelRects = {}
28182  g._morePanelY = panelY
28183  for (const item of items) {
28184    ctx.fillStyle = 'rgba(40,35,60,0.6)'
28185    R.rr(listX, itemY, listW, itemH - 4 * S, 8 * S); ctx.fill()
28186    ctx.fillStyle = '#e0d8c0'
28187    ctx.font = `${12*S}px "PingFang SC",sans-serif`
28188    ctx.textAlign = 'left'; ctx.textBaseline = 'middle'
28189    ctx.fillText(item.label, listX + 12 * S, itemY + (itemH - 4 * S) / 2)
28190    if (item.toggle !== null && item.toggle !== undefined) {
28191      const swW = 40 * S, swH = 22 * S
28192      const swX = listX + listW - swW - 8 * S
28193      const swY = itemY + (itemH - 4 * S) / 2 - swH / 2
28194      ctx.fillStyle = item.toggle ? 'rgba(77,204,77,0.8)' : 'rgba(100,100,120,0.5)'
28195      R.rr(swX, swY, swW, swH, swH / 2); ctx.fill()
28196      const knobR = swH / 2 - 2 * S
28197      const knobX = item.toggle ? swX + swW - swH / 2 : swX + swH / 2
28198      ctx.beginPath()
28199      ctx.arc(knobX, swY + swH / 2, knobR, 0, Math.PI * 2)
28200      ctx.fillStyle = '#fff'; ctx.fill()
28201      g._morePanelRects[item.key] = [listX, itemY, listW, itemH - 4 * S]
28202    } else {
28203      ctx.fillStyle = 'rgba(200,180,120,0.5)'
28204      ctx.font = `${14*S}px "PingFang SC",sans-serif`
28205      ctx.textAlign = 'right'
28206      ctx.fillText('', listX + listW - 12 * S, itemY + (itemH - 4 * S) / 2)
28207      g._morePanelRects[item.key] = [listX, itemY, listW, itemH - 4 * S]
28208    }
28209    itemY += itemH
28210  }
28211  if (g._morePanelRects.feedback) {
28212    g._feedbackBtnRect = g._morePanelRects.feedback
28213  }
28214  ctx.restore()
28215 }
28216 // ===== 宝箱浮钮（右下角，与模式切换左右对称） =====
28217 // ===== 宝箱浮钮（右上角，顶栏旁边）=====
28218 function drawChestBtn(g) {
28219  const { ctx, R, W, S, safeTop } = V
28220  const { getUnclaimedCount } = require('../data/chestConfig')
28221  // 图标尺寸：避开右上角系统按钮（...），下移至其下方
28222  const btnSz = 48 * S
28223  const btnX = W - btnSz - 10 * S
28224  const btnY = safeTop + 52 * S
28225  ctx.save()
```

```
28226  // 点击缩放动画：按下 → 弹起回弹
28227  const pressAge = g._chestPressTime ? (Date.now() - g._chestPressTime) : 9999
28228  let pressScale = 1.0
28229  if (pressAge < 120) {
28230    // 0~120ms：压下，scale 1→0.82
28231    pressScale = 1.0 - 0.18 * (pressAge / 120)
28232  } else if (pressAge < 300) {
28233    // 120~300ms：弹起回弹，用 sin 模拟弹性
28234    pressScale = 0.82 + 0.25 * Math.sin(Math.PI * (pressAge - 120) / 180)
28235  }
28236  const cx = btnX + btnSz / 2
28237  const cy = btnY + btnSz / 2
28238  ctx.translate(cx, cy)
28239  ctx.scale(pressScale, pressScale)
28240  ctx.translate(-cx, -cy)
28241  const chestImg = R.getImg('assets/ui/icon_chest.png')
28242  if (chestImg && chestImg.width > 0) {
28243    ctx.drawImage(chestImg, btnX, btnY, btnSz, btnSz)
28244  } else {
28245    // fallback：绘制圆形背景 + emoji
28246    ctx.fillStyle = 'rgba(20,18,38,0.75)'
28247    R.rr(btnX, btnY, btnSz, btnSz, 8 * S); ctx.fill()
28248    ctx.font = `${btnSz * 0.55}px sans-serif`
28249    ctx.textAlign = 'center'; ctx.textBaseline = 'middle'
28250    ctx.fillText('?', btnX + btnSz / 2, btnY + btnSz / 2)
28251  }
28252  // 红点 + 数字（右上角）
28253  const count = getUnclaimedCount(g.storage)
28254  if (count > 0) {
28255    const dotR = 8 * S
28256    const dotX = btnX + btnSz - dotR * 0.5
28257    const dotY = btnY + dotR * 0.5
28258    ctx.beginPath()
28259    ctx.arc(dotX, dotY, dotR, 0, Math.PI * 2)
28260    ctx.fillStyle = '#ff3333'
28261    ctx.fill()
28262    ctx.fillStyle = '#fff'
28263    ctx.font = `bold ${9*S}px "PingFang SC",sans-serif`
28264    ctx.textAlign = 'center'; ctx.textBaseline = 'middle'
28265    ctx.fillText(String(count), dotX, dotY)
28266  }
28267  g._chestBtnRect = [btnX, btnY, btnSz, btnSz]
28268  ctx.restore()
28269 }
28270 // ===== 开发者重置确认弹窗 =====
28271 function drawDevResetDialog(g) {
28272  if (!g._showDevResetDialog) return
28273  const { ctx, R, W, H, S } = V
28274  ctx.save()
28275  ctx.fillStyle = 'rgba(0,0,0,0.65)'
```

```
  ctx.fillRect(0, 0, W, H)
  const pw = W * 0.78, ph = 200 * S
  const px = (W - pw) / 2, py = (H - ph) / 2
  R.drawInfoPanel(px, py, pw, ph)
  ctx.textAlign = 'center'
  ctx.fillStyle = '#C07000'
  ctx.font = `bold ${15 * S}px "PingFang SC",sans-serif`
  ctx.fillText('? 开发者重置', W / 2, py + 36 * S)
  ctx.fillStyle = '#5a3a1a'
  ctx.font = `${12 * S}px "PingFang SC",sans-serif`
  ctx.fillText('将清除所有本地数据（存档/教学/漫画）', W / 2, py + 66 * S)
  ctx.fillText('并重启小游戏，确认继续？', W / 2, py + 86 * S)
  // 确认按钮
  const bw = pw * 0.42, bh = 40 * S
  const gap = 12 * S
  const confirmX = W / 2 + gap / 2
  const cancelX = W / 2 - gap / 2 - bw
  const btnY = py + ph - bh - 20 * S
  R.drawDialogBtn(confirmX, btnY, bw, bh, '一键重置', 'confirm')
  g._devResetConfirmRect = [confirmX, btnY, bw, bh]
  R.drawDialogBtn(cancelX, btnY, bw, bh, '取消', 'cancel')
  g._devResetCancelRect = [cancelX, btnY, bw, bh]
  ctx.restore()
}
// ===== 主入口 =====
function rTitle(g) {
  drawSceneArea(g)
  drawTopBar(g)
  drawStartBtn(g)
  drawModeSwitchBtn(g)
  drawChestBtn(g)
  drawSidebarBtn(g)
  drawBottomBar(g)
  drawAvatarWidget(g)
  drawStaminaBar(g)
  drawMorePanel(g)
  drawTitleStartDialog(g)
  drawSidebarPanel(g)
  drawDevResetDialog(g)
}
module.exports = { rTitle, getLayout, BAR_ITEMS, drawBottomBar }
/**
 * UI 公共工具函数 — 从各 View 中提取的重复代码
 */
const V = require('./env')
// ===== 渐变分隔线 =====
// color: RGB 字符串如 '201,168,76'; alpha: 中间段不透明度; fadeStart/fadeEnd: 渐变起止比例
function drawSeparator(c, x1, y, x2, color, alpha, fadeStart, fadeEnd, lineWidth) {
  color = color || '201,168,76'
  alpha = alpha != null ? alpha : 0.35
```

```
  fadeStart = fadeStart || 0.2
  fadeEnd = fadeEnd != null ? fadeEnd : (1 - fadeStart)
  const grad = c.createLinearGradient(x1, y, x2, y)
  grad.addColorStop(0, `rgba(${color},0)`)
  grad.addColorStop(fadeStart, `rgba(${color},${alpha})`)
  grad.addColorStop(fadeEnd, `rgba(${color},${alpha})`)
  grad.addColorStop(1, `rgba(${color},0)`)
  c.strokeStyle = grad; c.lineWidth = lineWidth || 1
  c.beginPath(); c.moveTo(x1, y); c.lineTo(x2, y); c.stroke()
}
// ===== 文本换行（返回行数组，按全角/半角字符宽度估算） =====
function wrapText(text, maxW, fontSize) {
  if (!text) return ['']
  const S = V.S
  const fullW = fontSize * S
  const halfW = fontSize * S * 0.55
  const result = []
  let line = '', lineW = 0
  for (let i = 0; i < text.length; i++) {
    const ch = text[i]
    const cw = ch.charCodeAt(0) > 127 ? fullW : halfW
    if (lineW + cw > maxW && line.length > 0) {
      result.push(line)
      line = ch; lineW = cw
    } else {
      line += ch; lineW += cw
    }
  }
  if (line) result.push(line)
  return result.length > 0 ? result : ['']
}
// ===== 文本换行并直接绘制（返回实际行数） =====
function wrapTextDraw(c, text, x, y, maxW, lineH) {
  let line = ''
  let lines = 1
  for (let i = 0; i < text.length; i++) {
    const ch = text[i]
    if (c.measureText(line + ch).width > maxW) {
      c.fillText(line, x, y)
      y += lineH
      line = ch
      lines++
    } else {
      line += ch
    }
  }
  if (line) c.fillText(line, x, y)
  return lines
}
// ===== 金色圆角按钮 =====
```

```
function drawGoldBtn(c, R, S, x, y, w, h, text, disabled, fontSize) {
  const r = h / 2
  const fs = fontSize || 14
  if (disabled) {
    c.fillStyle = 'rgba(80,70,50,0.6)'
    R.rr(x, y, w, h, r); c.fill()
    c.strokeStyle = '#666'; c.lineWidth = 1.5 * S
    R.rr(x, y, w, h, r); c.stroke()
    c.fillStyle = '#888'
    c.font = `bold ${fs*S}px "PingFang SC",sans-serif`
    c.textAlign = 'center'; c.textBaseline = 'middle'
    c.fillText(text, x + w / 2, y + h / 2)
    return
  }
  c.save()
  c.shadowColor = 'rgba(180,120,30,0.4)'; c.shadowBlur = 10 * S; c.shadowOffsetY = 3 * S
  const bg = c.createLinearGradient(x, y, x, y + h)
  bg.addColorStop(0, '#B8451A'); bg.addColorStop(0.5, '#9C3512'); bg.addColorStop(1, '#7A2A0E')
  c.fillStyle = bg
  R.rr(x, y, w, h, r); c.fill()
  c.restore()
  c.strokeStyle = '#D4A843'; c.lineWidth = 2 * S
  R.rr(x, y, w, h, r); c.stroke()
  c.save(); c.globalAlpha = 0.2
  const hl = c.createLinearGradient(x, y, x, y + h * 0.4)
  hl.addColorStop(0, '#fff'); hl.addColorStop(1, 'rgba(255,255,255,0)')
  c.fillStyle = hl
  R.rr(x + 2*S, y + 2*S, w - 4*S, h * 0.4, r); c.fill()
  c.restore()
  c.fillStyle = '#FFE8B8'
  c.font = `bold ${fs*S}px "PingFang SC",sans-serif`
  c.textAlign = 'center'; c.textBaseline = 'middle'
  c.shadowColor = 'rgba(0,0,0,0.5)'; c.shadowBlur = 4 * S
  c.fillText(text, x + w / 2, y + h / 2)
  c.shadowBlur = 0
}
// ===== 筛选后的宠物池 =====
function getFilteredPool(g) {
  const pool = g.storage.petPool || []
  const filter = g._petPoolFilter || 'all'
  if (filter === 'all') return pool
  return pool.filter(p => p.attr === filter)
}
// ===== 矩形命中检测 =====
function hitRect(x, y, rx, ry, rw, rh) {
  return x >= rx && x <= rx + rw && y >= ry && y <= ry + rh
}
module.exports = { drawSeparator, wrapText, wrapTextDraw, drawGoldBtn, getFilteredPool, hitRect }
const P = require('./platform')
/**
```

```
28426 * 平台原生按钮管理 — 授权按钮（排行榜入口）和意见反馈按钮
28427 * 支持微信/抖音双端
28428 */
28429 /** 更新 title 场景的透明授权按钮（覆盖在底部栏「排行」图标上） */
28430 function updateAuthBtn(g, dpr) {
28431  if (g.scene !== 'title') { g.storage.destroyUserInfoBtn(); return }
28432  if (g.storage.userAuthorized) { g.storage.destroyUserInfoBtn(); return }
28433  // 抖音端不需要透明按钮，通过 touchHandlers 中的点击触发 getUserProfile
28434  if (P.isDouyin) return
28435  // 「更多」面板或弹窗打开时，隐藏授权按钮避免遮挡
28436  if (g.showMorePanel || g.showNewRunConfirm) { g.storage.destroyUserInfoBtn(); return }
28437  const btnRect = g._rankBtnRect
28438  if (!btnRect) return
28439  const cssRect = { left: btnRect[0]/dpr, top: btnRect[1]/dpr, width: btnRect[2]/dpr, height: btn...
28440  if (!g.storage._userInfoBtn) {
28441    console.log('[AuthBtn] 创建透明授权按钮, cssRect:', JSON.stringify(cssRect))
28442    g.storage.showUserInfoBtn(cssRect, (ok, info) => {
28443      if (ok) console.log('[Ranking] 授权成功:', info.nickName, info.avatarUrl)
28444      else console.warn('[Ranking] 用户拒绝授权，以游客身份进入排行榜')
28445      g._openRanking()
28446    })
28447  }
28448 }
28449 /** 更新意见反馈按钮 — 仅在「更多」面板打开时显示，覆盖在「意见反馈」行上 */
28450 function updateFeedbackBtn(g, dpr) {
28451  if (g.scene !== 'title' || !g.showMorePanel) { destroyFeedbackBtn(g); return }
28452  const rect = g._feedbackBtnRect
28453  if (!rect) { destroyFeedbackBtn(g); return }
28454  const cssRect = { left: rect[0]/dpr, top: rect[1]/dpr, width: rect[2]/dpr, height: rect[3]/dpr }
28455  if (!g._feedbackBtn) {
28456    try {
28457      const btn = P.createFeedbackButton({
28458        type: 'text', text: '',
28459        style: { left: cssRect.left, top: cssRect.top, width: cssRect.width, height: cssRect.height,
28460          backgroundColor: 'rgba(0,0,0,0)', borderColor: 'rgba(0,0,0,0)', borderWidth: 0, borderR...
28461          color: 'rgba(0,0,0,0)', fontSize: 1, lineHeight: cssRect.height },
28462      })
28463      if (!btn) return
28464      g._feedbackBtn = btn; btn.show()
28465    } catch(e) { console.warn('[Feedback] createFeedbackButton 失败:', e) }
28466  } else {
28467    try { Object.assign(g._feedbackBtn.style, { left: cssRect.left, top: cssRect.top, width: cssR...
28468  }
28469 }
28470 function destroyFeedbackBtn(g) {
28471  if (g._feedbackBtn) {
28472    try { g._feedbackBtn.hide(); g._feedbackBtn.destroy() } catch(e) {}
28473    g._feedbackBtn = null
28474  }
28475 }
```

```
28476 module.exports = { updateAuthBtn, updateFeedbackBtn, destroyFeedbackBtn }
28477 // 云函数：获取用户openid
28478 const cloud = require('wx-server-sdk')
28479 cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV })
28480 exports.main = async (event, context) => {
28481   const wxContext = cloud.getWXContext()
28482   return {
28483     openid: wxContext.OPENID,
28484     appid: wxContext.APPID,
28485     unionid: wxContext.UNIONID
28486   }
28487 }
28488 // 云函数：自动创建所需数据库集合
28489 const cloud = require('wx-server-sdk')
28490 cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV })
28491 const COLLECTIONS = ['playerData', 'rankAll', 'rankDex', 'rankCombo']
28492 exports.main = async (event, context) => {
28493   const db = cloud.database()
28494   const created = []
28495   const existed = []
28496   const errors = []
28497   for (const name of COLLECTIONS) {
28498     try {
28499       await db.createCollection(name)
28500       created.push(name)
28501     } catch (e) {
28502       // 集合已存在（errCode可能是-501001或-501007）
28503       if (e.errCode === -501001 || e.errCode === -501007 || (e.message && (e.message.indexOf('alr...
28504         existed.push(name)
28505       } else {
28506         errors.push({ name, error: e.message || e.errMsg || String(e) })
28507       }
28508     }
28509   }
28510   return { created, existed, errors }
28511 }
28512 // 云函数：排行榜（提交分数 + 查询排行）
28513 // 支持3种排行：速通榜(all)、图鉴榜(dex)、连击榜(combo)
28514 const cloud = require('wx-server-sdk')
28515 cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV })
28516 const db = cloud.database()
28517 const _ = db.command
28518 exports.main = async (event, context) => {
28519   const wxContext = cloud.getWXContext()
28520   const openid = wxContext.OPENID
28521   const { action } = event
28522   // ===== 提交/更新分数（通关时调用）=====
28523   if (action === 'submit') {
28524     const t0 = Date.now()
28525     const { nickName, avatarUrl, floor, pets, weapon, totalTurns, petDexCount, maxCombo } = event
```

```
28526  if (!floor || floor <= 0) return { code: -1, msg: '无效层数' }
28527  const baseRecord = {
28528    uid: openid,
28529    nickName: nickName || '修士',
28530    avatarUrl: avatarUrl || '',
28531    floor,
28532    pets: (pets || []).slice(0, 5).map(p => ({ name: p.name, attr: p.attr })),
28533    weapon: weapon ? { name: weapon.name } : null,
28534    totalTurns: totalTurns || 0,
28535    timestamp: db.serverDate(),
28536  }
28537  try {
28538    // 同时用 _openid 和 uid 查找，避免漏查导致重复插入
28539    const existing = await db.collection('rankAll').where(_.or([
28540      { _openid: openid },
28541      { uid: openid }
28542    ])).get()
28543    console.log('[submit] 查已有记录:', existing.data.length, '条, 耗时', Date.now() - t0, 'ms')
28544    if (existing.data.length > 1) {
28545      // 去重：保留最佳记录，删除多余的
28546      const sorted = existing.data.sort((a, b) => {
28547        if (_isBetterScore(a.floor, a.totalTurns, b.floor, b.totalTurns)) return -1
28548        return 1
28549      })
28550      for (let i = 1; i < sorted.length; i++) {
28551        await db.collection('rankAll').doc(sorted[i]._id).remove()
28552      }
28553      const old = sorted[0]
28554      const isBetter = _isBetterScore(floor, totalTurns, old.floor, old.totalTurns)
28555      if (isBetter) {
28556        await db.collection('rankAll').doc(old._id).update({ data: baseRecord })
28557      } else {
28558        await db.collection('rankAll').doc(old._id).update({
28559          data: { nickName: baseRecord.nickName, avatarUrl: baseRecord.avatarUrl }
28560        })
28561      }
28562    } else if (existing.data.length === 1) {
28563      const old = existing.data[0]
28564      const isBetter = _isBetterScore(floor, totalTurns, old.floor, old.totalTurns)
28565      if (isBetter) {
28566        await db.collection('rankAll').doc(old._id).update({ data: baseRecord })
28567      } else {
28568        await db.collection('rankAll').doc(old._id).update({
28569          data: { nickName: baseRecord.nickName, avatarUrl: baseRecord.avatarUrl }
28570        })
28571      }
28572    } else {
28573      await db.collection('rankAll').add({ data: baseRecord })
28574    }
28575    // 同时更新图鉴榜和连击榜
```

```
28576     await _updateDexCombo(openid, nickName, avatarUrl, petDexCount, maxCombo)
28577     console.log('[submit] 完成, 总耗时', Date.now() - t0, 'ms')
28578     return { code: 0, msg: '提交成功', ms: Date.now() - t0 }
28579   } catch (e) {
28580     console.error('排行榜提交失败:', e, '耗时', Date.now() - t0, 'ms')
28581     return { code: -1, msg: e.message }
28582   }
28583 }
28584 // ===== 单独提交图鉴/连击（非通关时）=====
28585 if (action === 'submitDexCombo') {
28586   const { nickName, avatarUrl, petDexCount, maxCombo } = event
28587   try {
28588     await _updateDexCombo(openid, nickName || '修士', avatarUrl || '', petDexCount, maxCombo)
28589     return { code: 0, msg: '提交成功' }
28590   } catch (e) {
28591     return { code: -1, msg: e.message }
28592   }
28593 }
28594 // ===== 提交 + 查询一体化（减少云函数调用次数）=====
28595 if (action === 'submitAndGetAll') {
28596   const t0 = Date.now()
28597   const debug = { openid }
28598   try {
28599     // 1. 提交分数（如果有）
28600     const { nickName, avatarUrl, floor, pets, weapon, totalTurns, petDexCount, maxCombo } = event
28601     if (floor && floor > 0) {
28602       const baseRecord = {
28603         uid: openid,
28604         nickName: nickName || '修士',
28605         avatarUrl: avatarUrl || '',
28606         floor,
28607         pets: (pets || []).slice(0, 5).map(p => ({ name: p.name, attr: p.attr })),
28608         weapon: weapon ? { name: weapon.name } : null,
28609         totalTurns: totalTurns || 0,
28610         timestamp: db.serverDate(),
28611       }
28612       const existing = await db.collection('rankAll').where(_.or([
28613         { _openid: openid }, { uid: openid }
28614       ])).get()
28615       if (existing.data.length > 1) {
28616         const sorted = existing.data.sort((a, b) => _isBetterScore(a.floor, a.totalTurns, b.flo...
28617         for (let i = 1; i < sorted.length; i++) {
28618           await db.collection('rankAll').doc(sorted[i]._id).remove()
28619         }
28620         const old = sorted[0]
28621         if (_isBetterScore(floor, totalTurns, old.floor, old.totalTurns)) {
28622           await db.collection('rankAll').doc(old._id).update({ data: baseRecord })
28623         } else {
28624           await db.collection('rankAll').doc(old._id).update({ data: { nickName: baseRecord.nic...
28625         }
```

```
28626     } else if (existing.data.length === 1) {
28627       const old = existing.data[0]
28628       if (_isBetterScore(floor, totalTurns, old.floor, old.totalTurns)) {
28629         await db.collection('rankAll').doc(old._id).update({ data: baseRecord })
28630       } else {
28631         await db.collection('rankAll').doc(old._id).update({ data: { nickName: baseRecord.nic...
28632       }
28633     } else {
28634       await db.collection('rankAll').add({ data: baseRecord })
28635     }
28636     debug.submitMs = Date.now() - t0
28637     // 同时更新图鉴榜和连击榜（并行）
28638     await _updateDexCombo(openid, nickName || '修士', avatarUrl || '', petDexCount, maxCombo)
28639     debug.dexComboMs = Date.now() - t0
28640   }
28641   // 2. 拉取排行榜
28642   const getRes = await db.collection('rankAll')
28643     .orderBy('floor', 'desc')
28644     .limit(100)
28645     .get()
28646   debug.getCount = getRes.data.length
28647   debug.fetchMs = Date.now() - t0
28648   const records = getRes.data
28649   if (records.length === 0) {
28650     debug.totalMs = Date.now() - t0
28651     return { code: 0, list: [], myRank: -1, debug }
28652   }
28653   const deduped = _deduplicateByOpenid(records, _rankAllComparator)
28654   debug.dedupedCount = deduped.length
28655   const list = deduped.slice(0, 50)
28656   // 3. 从已拉到的数据中找自己排名
28657   let myRank = -1
28658   const idx = deduped.findIndex(r => (r._openid === openid || r.uid === openid))
28659   if (idx >= 0) {
28660     myRank = idx + 1
28661     debug.myFoundIn = 'cache'
28662   } else {
28663     // 不在前100条里
28664     const myRes = await db.collection('rankAll').where(_.or([
28665       { uid: openid }, { _openid: openid }
28666     ])).get()
28667     debug.myCount = myRes.data.length
28668     if (myRes.data.length > 0) {
28669       const my = myRes.data.sort((a, b) => _isBetterScore(a.floor, a.totalTurns, b.floor, b.t...
28670       const [higherFloor, sameFloorBetter] = await Promise.all([
28671         db.collection('rankAll').where({ floor: _.gt(my.floor) }).count(),
28672         my.totalTurns > 0
28673           ? db.collection('rankAll').where({ floor: _.eq(my.floor), totalTurns: _.gt(0).and(_...
28674           : Promise.resolve({ total: 0 })
28675       ])
```

```
28676        myRank = higherFloor.total + sameFloorBetter.total + 1
28677        debug.myFoundIn = 'query'
28678      }
28679    }
28680    debug.totalMs = Date.now() - t0
28681    return { code: 0, list, myRank, debug }
28682  } catch (e) {
28683    debug.totalMs = Date.now() - t0
28684    return { code: -1, msg: e.message, list: [], debug }
28685  }
28686 }
28687 // ===== 查询速通榜（总排行）=====
28688 if (action === 'getAll') {
28689  const debug = {}
28690  const t0 = Date.now()
28691  try {
28692    debug.openid = openid
28693    // 拉取排行榜（1次DB查询）
28694    const getRes = await db.collection('rankAll')
28695      .orderBy('floor', 'desc')
28696      .limit(100)
28697      .get()
28698    debug.getCount = getRes.data.length
28699    debug.fetchMs = Date.now() - t0
28700    const records = getRes.data
28701    if (records.length === 0) {
28702      debug.totalMs = Date.now() - t0
28703      return { code: 0, list: [], myRank: -1, debug }
28704    }
28705    // 构建排行列表
28706    const deduped = _deduplicateByOpenid(records, (a, b) => {
28707      if (a.floor !== b.floor) return b.floor - a.floor
28708      const aT = a.totalTurns || 0, bT = b.totalTurns || 0
28709      if (aT !== bT) {
28710        if (aT > 0 && bT > 0) return aT - bT
28711        if (aT > 0) return -1
28712        if (bT > 0) return 1
28713      }
28714      const aTime = a.timestamp ? new Date(a.timestamp).getTime() : 0
28715      const bTime = b.timestamp ? new Date(b.timestamp).getTime() : 0
28716      return bTime - aTime
28717    })
28718    debug.dedupedCount = deduped.length
28719    const list = deduped.slice(0, 50)
28720    // 先尝试从已有的 100 条数据中找到自己（避免额外查询）
28721    let myRank = -1
28722    let myRecord = records.find(r => r._openid === openid || r.uid === openid)
28723    if (myRecord) {
28724      // 直接从 deduped 列表中算排名
28725      const idx = deduped.findIndex(r => (r._openid === openid || r.uid === openid))
```

```
28726      if (idx >= 0) {
28727        myRank = idx + 1
28728      } else {
28729        // 数据在 100 条内但被去重掉了，用 count 查
28730        const [higherFloor, sameFloorBetter] = await Promise.all([
28731          db.collection('rankAll').where({ floor: _.gt(myRecord.floor) }).count(),
28732          myRecord.totalTurns > 0
28733            ? db.collection('rankAll').where({
28734                floor: _.eq(myRecord.floor),
28735                totalTurns: _.gt(0).and(_.lt(myRecord.totalTurns))
28736              }).count()
28737            : Promise.resolve({ total: 0 })
28738        ])
28739        myRank = higherFloor.total + sameFloorBetter.total + 1
28740      }
28741      debug.myFoundIn = 'cache'
28742    } else {
28743      // 不在前 100 条里，需要额外查询（1次DB）
28744      const myRes = await db.collection('rankAll').where(_.or([
28745        { uid: openid },
28746        { _openid: openid }
28747      ])).get()
28748      debug.myCount = myRes.data.length
28749      if (myRes.data.length > 0) {
28750        const my = myRes.data.sort((a, b) => {
28751          if (_isBetterScore(a.floor, a.totalTurns, b.floor, b.totalTurns)) return -1
28752          return 1
28753        })[0]
28754        // 两次 count 并行查询
28755        const [higherFloor, sameFloorBetter] = await Promise.all([
28756          db.collection('rankAll').where({ floor: _.gt(my.floor) }).count(),
28757          my.totalTurns > 0
28758            ? db.collection('rankAll').where({
28759                floor: _.eq(my.floor),
28760                totalTurns: _.gt(0).and(_.lt(my.totalTurns))
28761              }).count()
28762            : Promise.resolve({ total: 0 })
28763        ])
28764        myRank = higherFloor.total + sameFloorBetter.total + 1
28765        debug.myFoundIn = 'query'
28766      }
28767    }
28768    debug.totalMs = Date.now() - t0
28769    return { code: 0, list, myRank, debug }
28770  } catch (e) {
28771    debug.totalMs = Date.now() - t0
28772    return { code: -1, msg: e.message, list: [], debug, stack: e.stack }
28773  }
28774 }
28775 // ===== 查询图鉴榜 =====
```

```
28776 if (action === 'getDex') {
28777   try {
28778     const res = await db.collection('rankDex')
28779       .orderBy('petDexCount', 'desc')
28780       .limit(100)
28781       .get()
28782     const deduped = _deduplicateByOpenid(res.data, (a, b) => {
28783       if ((b.petDexCount || 0) !== (a.petDexCount || 0)) return (b.petDexCount || 0) - (a.petDe...
28784       const aTime = a.timestamp ? new Date(a.timestamp).getTime() : 0
28785       const bTime = b.timestamp ? new Date(b.timestamp).getTime() : 0
28786       return bTime - aTime
28787     })
28788     const list = deduped.slice(0, 50)
28789     let myRank = -1
28790     const myRes = await db.collection('rankDex').where(_.or([
28791       { uid: openid },
28792       { _openid: openid }
28793     ])).get()
28794     if (myRes.data.length > 0) {
28795       const myCount = myRes.data.sort((a, b) => (b.petDexCount || 0) - (a.petDexCount || 0))[0]...
28796       const betterCount = await db.collection('rankDex').where({ petDexCount: _.gt(myCount) }).....
28797       myRank = betterCount.total + 1
28798     }
28799     return { code: 0, list, myRank }
28800   } catch (e) {
28801     return { code: -1, msg: e.message, list: [] }
28802   }
28803 }
28804 // ===== 查询连击榜 =====
28805 if (action === 'getCombo') {
28806   try {
28807     const res = await db.collection('rankCombo')
28808       .orderBy('maxCombo', 'desc')
28809       .limit(100)
28810       .get()
28811     const deduped = _deduplicateByOpenid(res.data, (a, b) => {
28812       if ((b.maxCombo || 0) !== (a.maxCombo || 0)) return (b.maxCombo || 0) - (a.maxCombo || 0)
28813       const aTime = a.timestamp ? new Date(a.timestamp).getTime() : 0
28814       const bTime = b.timestamp ? new Date(b.timestamp).getTime() : 0
28815       return bTime - aTime
28816     })
28817     const list = deduped.slice(0, 50)
28818     let myRank = -1
28819     const myRes = await db.collection('rankCombo').where(_.or([
28820       { uid: openid },
28821       { _openid: openid }
28822     ])).get()
28823     if (myRes.data.length > 0) {
28824       const myCombo = myRes.data.sort((a, b) => (b.maxCombo || 0) - (a.maxCombo || 0))[0].maxCombo
28825       const betterCount = await db.collection('rankCombo').where({ maxCombo: _.gt(myCombo) }).c...
```

```
28826        myRank = betterCount.total + 1
28827      }
28828      return { code: 0, list, myRank }
28829    } catch (e) {
28830      return { code: -1, msg: e.message, list: [] }
28831    }
28832  }
28833  return { code: -1, msg: '未知操作' }
28834 }
28835 // 速通榜排序比较器
28836 function _rankAllComparator(a, b) {
28837  if (a.floor !== b.floor) return b.floor - a.floor
28838  const aT = a.totalTurns || 0, bT = b.totalTurns || 0
28839  if (aT !== bT) {
28840    if (aT > 0 && bT > 0) return aT - bT
28841    if (aT > 0) return -1
28842    if (bT > 0) return 1
28843  }
28844  const aTime = a.timestamp ? new Date(a.timestamp).getTime() : 0
28845  const bTime = b.timestamp ? new Date(b.timestamp).getTime() : 0
28846  return bTime - aTime
28847 }
28848 // 按用户去重：优先用 _openid，没有则用 uid 字段，都没有则用 _id（不跳过）
28849 function _deduplicateByOpenid(records, compareFn) {
28850  const map = {}
28851  for (const r of records) {
28852    const key = r._openid || r.uid || r._id
28853    if (!key) continue
28854    if (!map[key]) {
28855      map[key] = r
28856    } else {
28857      if (compareFn(r, map[key]) < 0) {
28858        map[key] = r
28859      }
28860    }
28861  }
28862  return Object.values(map).sort(compareFn)
28863 }
28864 // 判断新成绩是否优于旧成绩
28865 function _isBetterScore(newFloor, newTurns, oldFloor, oldTurns) {
28866  // 通关 > 未通关
28867  if (newFloor >= 30 && oldFloor < 30) return true
28868  if (newFloor < 30 && oldFloor >= 30) return false
28869  // 都通关：有回合数的优于没回合数的，都有则回合数更少更好
28870  if (newFloor >= 30 && oldFloor >= 30) {
28871    if (newTurns > 0 && oldTurns > 0) return newTurns < oldTurns
28872    if (newTurns > 0) return true
28873    return false
28874  }
28875  // 都未通关：层数更高更好
```

```
28876  return newFloor > oldFloor
28877 }
28878 // 更新图鉴榜和连击榜（带去重逻辑）
28879 async function _updateDexCombo(openid, nickName, avatarUrl, petDexCount, maxCombo) {
28880  // 图鉴榜
28881  if (petDexCount > 0) {
28882   const dexExist = await db.collection('rankDex').where(_.or([
28883    { _openid: openid },
28884    { uid: openid }
28885   ])).get()
28886   const dexRecord = {
28887    uid: openid, nickName, avatarUrl, petDexCount,
28888    timestamp: db.serverDate(),
28889   }
28890   if (dexExist.data.length > 1) {
28891    // 去重
28892    const sorted = dexExist.data.sort((a, b) => (b.petDexCount || 0) - (a.petDexCount || 0))
28893    for (let i = 1; i < sorted.length; i++) {
28894     await db.collection('rankDex').doc(sorted[i]._id).remove()
28895    }
28896    if (petDexCount >= (sorted[0].petDexCount || 0)) {
28897     await db.collection('rankDex').doc(sorted[0]._id).update({ data: dexRecord })
28898    } else {
28899     await db.collection('rankDex').doc(sorted[0]._id).update({ data: { nickName, avatarUrl } })
28900    }
28901   } else if (dexExist.data.length === 1) {
28902    if (petDexCount >= (dexExist.data[0].petDexCount || 0)) {
28903     await db.collection('rankDex').doc(dexExist.data[0]._id).update({ data: dexRecord })
28904    } else {
28905     await db.collection('rankDex').doc(dexExist.data[0]._id).update({
28906      data: { nickName, avatarUrl }
28907     })
28908    }
28909   } else {
28910    await db.collection('rankDex').add({ data: dexRecord })
28911   }
28912  }
28913  // 连击榜
28914  if (maxCombo > 0) {
28915   const comboExist = await db.collection('rankCombo').where(_.or([
28916    { _openid: openid },
28917    { uid: openid }
28918   ])).get()
28919   const comboRecord = {
28920    uid: openid, nickName, avatarUrl, maxCombo,
28921    timestamp: db.serverDate(),
28922   }
28923   if (comboExist.data.length > 1) {
28924    // 去重
28925    const sorted = comboExist.data.sort((a, b) => (b.maxCombo || 0) - (a.maxCombo || 0))
```

```
28926      for (let i = 1; i < sorted.length; i++) {
28927        await db.collection('rankCombo').doc(sorted[i]._id).remove()
28928      }
28929      if (maxCombo > (sorted[0].maxCombo || 0)) {
28930        await db.collection('rankCombo').doc(sorted[0]._id).update({ data: comboRecord })
28931      } else {
28932        await db.collection('rankCombo').doc(sorted[0]._id).update({ data: { nickName, avatarUrl ...
28933      }
28934    } else if (comboExist.data.length === 1) {
28935      if (maxCombo > (comboExist.data[0].maxCombo || 0)) {
28936        await db.collection('rankCombo').doc(comboExist.data[0]._id).update({ data: comboRecord })
28937      } else {
28938        await db.collection('rankCombo').doc(comboExist.data[0]._id).update({
28939          data: { nickName, avatarUrl }
28940        })
28941      }
28942    } else {
28943      await db.collection('rankCombo').add({ data: comboRecord })
28944    }
28945  }
28946 }
28947 // 云函数：每日/每周重置（定时触发 + 主动调用）
28948 const cloud = require('wx-server-sdk')
28949 cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV })
28950 const db = cloud.database()
28951 const _ = db.command
28952 exports.main = async (event, context) => {
28953   const now = new Date()
28954   // UTC+8时区
28955   const cnNow = new Date(now.getTime() + 8 * 60 * 60 * 1000)
28956   const today = `${cnNow.getFullYear()}-${cnNow.getMonth()+1}-${cnNow.getDate()}`
28957   const day = cnNow.getDay() // 0=周日，1=周一
28958   try {
28959     // 如果是定时触发，批量重置所有玩家
28960     if (!event.openid) {
28961       const allPlayers = await db.collection('playerData').limit(1000).get()
28962       let resetCount = 0
28963       for (const player of allPlayers.data) {
28964         const updates = {}
28965         // 每日任务重置
28966         if (!player.dailyTask || player.dailyTask.lastReset !== today) {
28967           updates.dailyTask = {
28968             tasks: [false, false, false],
28969             rewardGot: false,
28970             lastReset: today,
28971             comboChallengeCount: 0,
28972             levelChallengeCount: 0
28973           }
28974         }
28975         // 每周一重置周回挑战
```

```
28976       if (day === 1 && (!player.weeklyChallenge || player.weeklyChallenge.lastReset !== today)) {
28977         updates.weeklyChallenge = {
28978           count: 3,
28979           bestCombo: 0,
28980           lastReset: today,
28981           weeklyLevelId: Math.floor(Math.random() * 8) + 1
28982         }
28983       }
28984       if (Object.keys(updates).length > 0) {
28985         await db.collection('playerData').doc(player._id).update({ data: updates })
28986         resetCount++
28987       }
28988     }
28989     return { code: 0, msg: `批量重置完成，重置${resetCount}个玩家` }
28990   }
28991   // 单个玩家主动调用
28992   const openid = event.openid
28993   const res = await db.collection('playerData').where({ openid }).get()
28994   if (res.data.length === 0) return { code: -1, msg: '玩家数据不存在' }
28995   const player = res.data[0]
28996   const updates = {}
28997   // 每日任务重置
28998   if (!player.dailyTask || player.dailyTask.lastReset !== today) {
28999     updates.dailyTask = {
29000       tasks: [false, false, false],
29001       rewardGot: false,
29002       lastReset: today,
29003       comboChallengeCount: 0,
29004       levelChallengeCount: 0
29005     }
29006   }
29007   // 每周一重置周回挑战
29008   if (day === 1 && (!player.weeklyChallenge || player.weeklyChallenge.lastReset !== today)) {
29009     updates.weeklyChallenge = {
29010       count: 3,
29011       bestCombo: 0,
29012       lastReset: today,
29013       weeklyLevelId: Math.floor(Math.random() * 8) + 1
29014     }
29015   }
29016   if (Object.keys(updates).length > 0) {
29017     await db.collection('playerData').doc(player._id).update({ data: updates })
29018     Object.assign(player, updates)
29019   }
29020   return { code: 0, msg: '重置成功', progress: player }
29021 } catch (err) {
29022   console.error('云函数重置失败：', err)
29023   return { code: -1, msg: '重置失败', err: err.message }
29024 }
29025 }
```